

INSLIB

Software Documentation and Mathematical Models



Jan Zwiener
jan@zwiener.org

Version 1.2.0
October 3, 2026

Abstract

INSLIB is a portable C-only (C11) library for 3D navigation state estimation. Its core is a 15-state (optionally 18-state) error-state Kalman filter fusing inertial measurements with (delayed) GNSS position/velocity, barometer, magnetometer, local position references, absolute yaw references, scalar ground speed, ranges to known anchors and zero-velocity / zero-rotation information. Around it the library provides two standalone attitude filters (ARS/AHRS), a baro/accelerometer vertical channel filter (`baro_alt`), a World Magnetic Model (WMM) lookup and a wrapper (`nav_suite`) that runs all filters in parallel with graceful degradation. All covariances are maintained in UDU (“square-root”) factorisation (Bierman/Thornton). The code uses no heap and no OS services, and its per-epoch execution time is bounded. This document describes the software architecture, the design decisions and the mathematical models implemented.

Contents

1	Introduction	4
1.1	Scope and goals	4
1.2	Module overview	4
2	Conventions and notation	5
3	Software architecture	5
3.1	Structure	5
3.2	Data flow: one measurement epoch	5

3.3	Parallel redundant filters and mode arbitration	6
3.4	Robustness policy	7
3.5	Memory and timing design	10
3.6	Sensor noise defaults	10
4	Estimation backend: UDU factorisation	10
5	INS: the navigation ESKF	11
5.1	State definition	11
5.2	Strapdown mechanisation	12
5.3	Incremental absolute-position bookkeeping	13
5.4	Covariance prediction	13
5.5	Measurement models	13
5.5.1	GNSS position	13
5.5.2	Barometric height (ins-internal)	14
5.5.3	GNSS velocity	14
5.5.4	GNSS covariance conditioning	15
5.5.5	Absolute speed (odometry, OBD-II)	15
5.5.6	Ranges to known anchors	16
5.5.7	Position decimation against an unknown correlation	17
5.5.8	Local NED position	18
5.5.9	Absolute yaw	18
5.5.10	Magnetometer (yaw-only vector fusion)	18
5.5.11	Zero-velocity and zero-rotation updates (ZUPT/ZARU)	20
5.6	Delayed measurements: the history buffer	20
5.7	Auto-initialisation	20
5.8	Automatic ZUPT/ZARU detection	21
5.8.1	Tuning the detector for a platform	23
5.9	Dead reckoning, re-acquisition, readiness	24
5.10	Vertical origin relocation	25
6	AHRS: standalone attitude filters	26
6.1	Model	26
6.2	Accelerometer leveling	26
6.3	Magnetometer heading (scalar fusion)	27
6.4	True-north referencing (declination step)	27
7	Vertical channel: height systems, baro_alt and the offset filter	28
7.1	Height systems	28
7.2	baro_alt: 3-state baro/accelerometer filter	30
7.3	local_gnss_alt: 1-state offset filter	31
8	Frame offsets and coordinate conversion	32
9	World Magnetic Model lookup	33
9.1	Declination is a wrapped angle	33
9.2	Dip pole exclusion zones	33
10	Design decisions	34

11 Dataset configuration: config.yaml	36
11.1 Parameter cross-reference: config.yaml ↔ C API	50
11.2 Free-inertial mode (free_inertial_start)	55
11.3 Dataset CSV streams	57
11.4 replay.py analysis options (command line)	58
12 Verification environment (overview)	59

1 Introduction

1.1 Scope and goals

INSLIB targets embedded navigation applications (vehicles, aircraft, robots, indoor platforms) with aerospace-style engineering constraints:

- **Portable C11.** No OS dependencies, no compiler extensions.
- **No heap/dynamic memory.** All state exists in caller-provided structs, the library never calls `malloc/free`.
- **Bounded execution time (WCET).** No unbounded loops, recursion, or input-dependent iteration counts in the per-epoch path.
- **Single-precision hot path.** Strapdown, prediction and fusion run in 32-bit IEEE 754 float. 64-bit double is limited to coordinate calculations that require a higher precision.
- **Fail-operational robustness.** Non-finite inputs (NaN, Inf) are dropped at the API boundary, implausible measurements are statistically tracked, and rejections are counted in diagnostics.

Development follows a requirements-first process: behaviour is captured in a machine-checked requirements database (`requirements/`, checked by `make reqs`) and verified by unit/integration tests (`make test`) and replays of real datasets (`make datasets`). This document does not restate the requirements, it explains the architecture, the mathematics, and the tuning parameters.

1.2 Module overview

Module	Files	Purpose
INS	<code>ins.[ch]</code>	15/18-state Error-State Kalman Filter: position, velocity, attitude, IMU biases, optional magnetometer hard-iron bias.
ahrs	<code>ahrs.[ch]</code>	Standalone 5/6-state attitude filter (ARS: roll/pitch, AHRS: roll/pitch/yaw with magnetometer heading).
baro_alt	<code>baro_alt.[ch]</code>	3-state baro/accelerometer vertical channel filter, plus <code>local_gnss_alt_t</code> , a 1-state local-height-to-ellipsoid offset filter.
nav_suite	<code>nav_suite.[ch]</code>	Wrapper: runs INS + two AHRS instances + vertical filters in parallel, mode arbitration, vertical datum alignment, best-source accessors.
geodetic_toolbox	<code>geodetic_toolbox.[ch]</code>	Quaternion algebra, WGS84 geodesy, gravity model, Earth/transport rate. Reusable outside INSLIB.
magnetic_model	<code>magnetic_model.[ch]</code> , <code>wmm_lut.h</code>	World Magnetic Model lookup (declination, inclination, field strength, NED reference vector) from a generated grid. Reusable.
sensor_defaults	<code>sensor_defaults.h</code>	Shared consumer-MEMS gyro noise defaults for INS and AHRS (see §3.6).
KFCore	submodule	<code>linalg</code> (column-major matrix functions), <code>kalman_udu</code> (Bierman/Thornton UDU routines).

2 Conventions and notation

- **Body frame (b):** FRD – x forward, y right, z down, right-handed.
- **Navigation frame (n):** NED – north, east, down, local-level frame.
- **Earth frame (e):** ECEF (WGS84).
- **Attitude:** Hamilton quaternion $\mathbf{q}_b^n$ with the scalar part first ($q_0 = w$), representing the body-to-NED rotation $\mathbf{R}_b^n$. Euler angles are Tait–Bryan ZYX (roll ϕ , pitch θ , yaw ψ).
- **Matrices** are stored column-major (BLAS/LAPACK convention).
- **Timestamps** are `int64` microseconds, monotonic.
- $[\mathbf{a}]_\times$ denotes the skew-symmetric cross-product matrix, $[\mathbf{a}]_\times \mathbf{b} = \mathbf{a} \times \mathbf{b}$.
- $\mathbf{f}^b$ is the accelerometer-measured specific force, $\boldsymbol{\omega}_{ib}^b$ the gyroscope-measured angular rate, $\mathbf{g}^n$ the local gravity (including the centrifugal term), $\mathbf{m}^n$ the magnetic reference field.

Units are SI throughout: m, m s^{-1} , rad, rad s^{-1} , m s^{-2} , Pa, magnetometer quantities in μT (matching the World Magnetic Model).

3 Software architecture

3.1 Structure

Figure 1 shows the module dependency structure. The three estimation modules (INS, AHRS, `baro_alt`) are independent of each other, each can be used standalone. They share only the math layer (`geodetic_toolbox`, `magnetic_model`, `KFCore`) and, for INS/AHRS, the IMU noise fallback constants (`sensor_defaults.h`). The `nav_suite` wrapper is the only module that knows all filters.

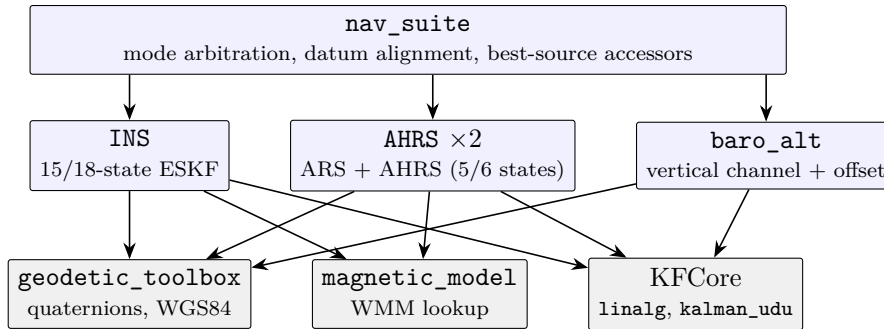


Figure 1: Module dependencies. Grey modules are (reusable) helper modules: `geodetic_toolbox`, `KFCore`, and `magnetic_model` are reusable outside this repository.

3.2 Data flow: one measurement epoch

All measurements of one epoch are collected in a single `ins_measurements_t` bundle (every field optional, marked by an `is_valid` flag) and pushed to `nav_suite_update` (or `ins_update` when using the core filter alone). Inside `ins_update` the processing order is fixed:

1. **Input sanitisation:** every consumed field is checked for NaN/Inf before any arithmetic (§3.4).

2. **Time-jump handling:** backward steps and oversized forward gaps are classified and handled (drop / reset / skip), a backward step that *persists* is read as a restarted timestamp source (§3.4).
3. **Strapdown integration** of the nominal state at IMU rate (§5.2).
4. **Covariance prediction**, throttled to the configured Kalman period rather than the IMU rate (§5.4).
5. **Measurement fusion** in fixed order: magnetometer, absolute yaw, zero-velocity, zero-rotation, local position, GNSS position/velocity, barometric height, absolute speed, ranges to known anchors, GNSS course-over-ground yaw.
6. **History snapshot** into a ring buffer (for delayed measurements, §5.6) and **health check**.

The wrapper additionally feeds timestamp + accelerometer + gyroscope (+ magnetometer) to the two AHRS instances, timestamp + accelerometer + attitude + pressure to the vertical filter, and baro/GNSS pairs to the offset filter.

3.3 Parallel redundant filters and mode arbitration

The suite runs redundant estimators of different complexity:

- **INS** needs position aiding to stay observable. It provides the full (3D) navigation solution.
- **AHRS/ARS** consume only IMU (+mag) and therefore keep producing attitude through arbitrary position-aiding outages. They serve as cross-check and fallback for the INS attitude.
- **baro_alt** plays the same role for the vertical channel: height and climb rate from baro + accelerometer only.

Height-source selection. If a barometer was sending measurements when INS started *on* GNSS, INS fuses the barometric height directly as its own vertical position measurement (§5.5.2) instead of the vertical component of the GNSS position. GNSS position fusion then supplies only the horizontal (N, E) rows and is graded only on its horizontal accuracy (§5.5.1), GNSS velocity is unaffected on all three axes. The selection targets GNSS’s comparatively poor vertical accuracy specifically: a local-position bootstrap keeps its own, typically better vertical reference regardless of barometer presence, and **baro_height_disable** suppresses the selection outright (§5.7).

The effect on INS’s *own* vertical state is that it no longer depends on GNSS at all, so it neither steps nor degrades at the FULL/COASTING/ATTITUDE_ONLY boundary – only the horizontal solution does. That is a statement about **ins**’s internal state, not about what the suite reports: the height accessors still prefer **baro_alt** outside FULL mode (§7.1), which is what keeps a height available when **ins** itself is not. **baro_alt** runs in parallel regardless, as a genuinely independent barometric estimate with its own state and accelerometer bias (§7.2). Without a barometer, **ins** fuses the full 3D GNSS position and the vertical channel follows the same mode machinery as the horizontal one. See §10 for the rationale.

nav_suite_get_mode() arbitrates a solution mode: FULL (ins ready, position aiding within the last 2s) → COASTING (ins ready but dead-reckoning) → ATTITUDE_ONLY (ins unavailable, AHRS attitude valid) → NONE. Accessors such as **nav_suite_get_rpy()** and **nav_suite_get_height()** always return the best available source, so a consumer keeps receiving a continuous output through outages. The attitude fallback order is ins → magnetometer-AHRS → ARS.

Heading across an ins re-initialisation. A quality-loss re-arm tears the `ins` instance down and re-derives its attitude at the next bootstrap. With a magnetometer that is unproblematic (the AHRS hands over a measured heading); without one, `ins` has no heading source and starts from the “heading unknown” prior ($\sigma \approx 180^\circ$). The ARS, however, kept running through the outage. Its yaw is not a heading – it free-integrates and drifts – but the *change* it reports is good over the timescale of an outage, and a change is all a re-init needs.

The wrapper therefore latches the pair (`ins` heading, ARS yaw) on every epoch `ins` is ready and confident about its heading ($\sigma \leq 15^\circ$), and offers the re-initialising filter the latched heading advanced by the yaw the ARS has turned since. Its uncertainty is the latched σ with the ARS’s accumulated drift added in quadrature (its z gyro-bias σ integrated over the elapsed time, inflated $10\times$ like every ARS-derived hint value); past 30° the carried heading is withheld and the “unknown” prior applies again. A re-bootstrapping ARS drops the carry-over outright, its yaw having restarted from the config seed.

`nav_suite_get_rpy()` reports the same reconstruction for yaw whenever it falls back to the ARS, so the suite’s heading does not jump to the ARS’s seed-relative yaw when `ins` drops out. Roll and pitch stay the ARS’s own. The per-filter accessor `nav_suite_get_rpy_ars()` keeps reporting the raw ARS yaw, being what a cross-check between the two estimators has to read.

Figure 2 shows the mode transitions, Figure 3 the resulting behaviour of the parallel filters across a GNSS outage: attitude and – if barometer data is available – height remain continuously available while the `ins` position degrades and recovers.

All local heights share one vertical datum (the `ins` NED origin). Whichever filter initialises first defines it: if `ins` is ready when the first pressure sample arrives, `baro_alt` anchors at the current NED height. If `baro_alt` was running first (auto-init with a late first fix), the `ins` origin is shifted vertically at bootstrap (§5.10) so the local heights agree, while the absolute solution is untouched. This alignment is what keeps `ins`’s own barometric height (once selected as the height source) and `baro_alt`’s independently-computed height consistent from the first shared epoch onward, the two still carry separate accelerometer-bias states and are not expected to stay bit-identical over a long run, only to track the same physical quantity closely. Since the height accessors hand over between exactly these two estimates at a mode change, the suite does not assume they stay close – it compares them every epoch and warns on a persistent gap, alongside the same cross-check on the vertical velocity (the two catch different failures: a divergence too slow to trip the velocity bound still integrates into an unbounded height gap).

3.4 Robustness policy

Three layers protect the state estimate, uniformly across all filters, following common navigation filter best practices [7]:

1. **NaN/Inf boundary screening.** Every comparison with NaN is false, so NaN passes through gates written as `if (x <= limit) reject;`. One corrupt sample would poison the state and force the health check to shut the filter down. Therefore all inputs are checked with `isfinite()` at the API boundary; corrupt blocks are invalidated and counted (`n_invalid_input`), processing continues. Finiteness is not the only way an input can be wrong: absolute headings (the yaw measurement and the attitude hint) are accepted in any single-turn convention – $[-\pi, \pi]$, $[0, 2\pi)$ – and normalised to $[-\pi, \pi]$, whereas a heading beyond one full turn is a unit or unwrapping error (degrees written into a radian field, an accumulated course) and is rejected and counted rather than wrapped into a plausible-looking but wrong heading.
2. **Statistical screening with two policies.** Each scalar measurement row is tested with a Mahalanobis (chi-square) gate inside the UDU update (§4). *Persistent absolute references*

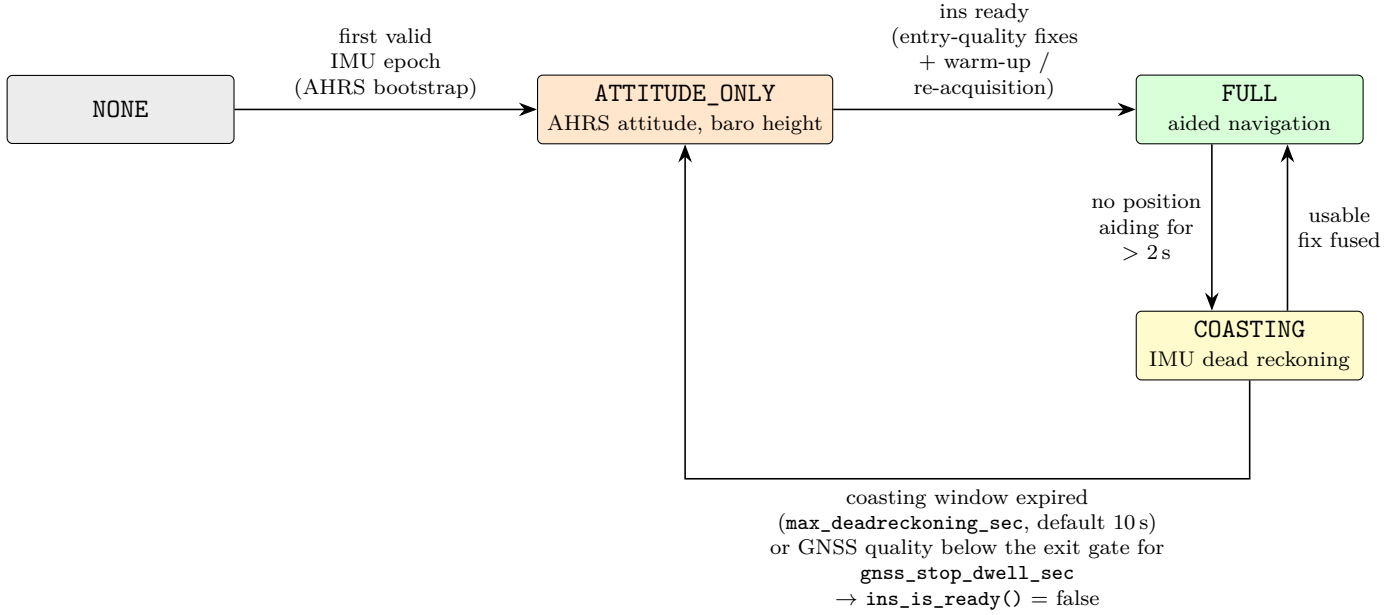


Figure 2: Solution-mode state machine (`nav_suite_get_mode`). The way out of **ATTITUDE_ONLY** depends on which cause put it there, and both require the fix stream to satisfy the entry-quality gate and its dwell (`gnss_start_max_*`, `gnss_init_dwell_sec`). After an expired coasting window the first usable fix *re-acquires*: it re-anchors position and velocity while attitude and IMU biases are kept, and the suite returns directly to **FULL** with no warm-up. After a quality loss the instance was torn down, so the return is a full re-bootstrap – new position and velocity, attitude re-seeded from the ARS/AHRS (heading from the magnetometer-AHRS, or from the pre-outage heading advanced by the ARS, §3.3), IMU biases carried over with inflated uncertainty – and readiness waits out the usual warm-up. The n-frame origin is carried over too, so local NED coordinates mean the same thing before and after the outage. Only a fix the platform could not have reached while unobserved – further from the last known position than a hard speed bound allows over the elapsed time – falls back to anchoring a fresh origin. Distance to the origin does not enter that decision: it is just how far the mission has travelled. `allow_unlimited_deadreckoning` disables the coasting-window cause; `gnss_stop_disable` disables the quality-loss cause, and `auto_reacquire_disable` disables the tear-down, falling back to holding `ins_is_ready()` down while the filter runs on.

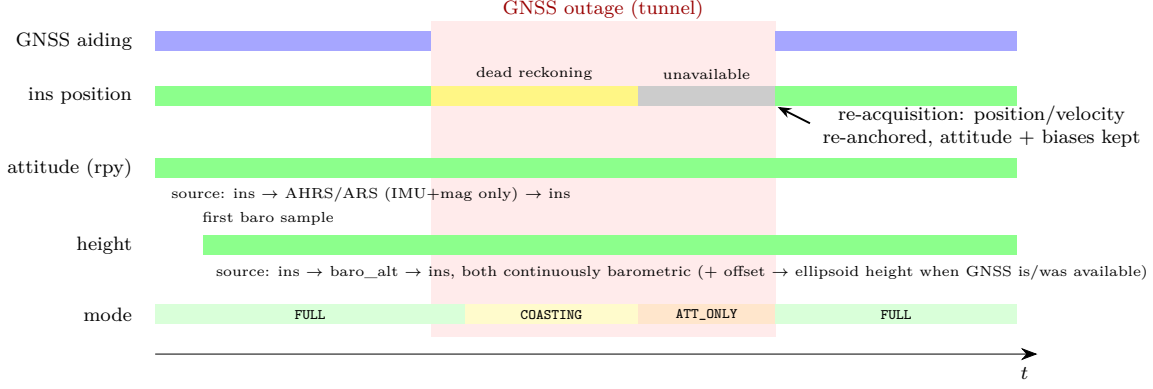


Figure 3: Graceful degradation across a GNSS outage. The AHRs/ARS filters (IMU+mag only) and the baro_alt filter (baro+accel+attitude) keep running unconditionally, so attitude and – with barometer data – height stay available throughout. Both switch sources across the outage (attitude: ins $\rightarrow$ AHRs/ARS $\rightarrow$ ins, height: ins $\rightarrow$ baro_alt $\rightarrow$ ins, since the accessors prefer ins only in FULL mode). The barometric height source does not remove that switch, it makes it benign: both sides are then the same barometric quantity on the same vertical datum rather than a GNSS-derived height handing over to a barometric one, so the remaining step is the disagreement between two independent estimators of one quantity – which the suite cross-checks continuously. ins’s horizontal position coasts on the IMU, degrades after the coasting window, and re-acquires with the first fix.

(GNSS, magnetometer, absolute yaw, barometer) are *downweighted* – their noise is inflated so the gate is met – never skipped: a skip policy would deadlock when a persistent offset exceeds an optimistic prior (the measurement is rejected forever while the covariance never shrinks). *High-rate local references* (lighthouse, UWB, mocap) are *skipped* on outliers: they glitch (occlusion, reflections) and a dropped sample is cheap at their rate.

3. **Health check.** After each epoch the state and the covariance factors are checked for finiteness and non-negative variances. A tripped check marks the filter uninitialised instead of publishing corrupt values; in the suite the AHRs and baro filters then re-bootstrap autonomously from the stream.

A fourth case is neither a corrupt value nor an outlier but a broken *timebase*: the sensor source reboots and its timestamp counter starts over, so every following epoch is stamped far in the past. Each is correctly dropped as older than `INS_MAX_DELAY_MS`, but the drop happens before the prediction step, so the internal time reference stays in the abandoned timebase, the next epoch measures the same negative delta, and the filter stays inert forever while still being fed good data. The filter therefore counts *consecutive* drops of this kind and, once the run reaches `INS_TIME_RESTART_EPOCHS`, forces the same defined reset as an oversized forward jump and re-arms into the collecting state (`n_time_restart_reset`). Any epoch inside the tolerated depth clears the run, which separates a restart from ordinary reordering: reordering arrives in bursts, a restart never ends. Unlike a forward jump, this reset is not suppressed by `allow_unlimited_deadreckoning` – coasting would wait for a timebase that never returns.

Drivers should not let it get that far. A driver reconstructing a 64-bit timeline from a narrower counter (as `tools/insrcv.c` does from a 32-bit microsecond source) can tell an overflow from a restart by *where* in its range the counter was, and stitch a restart onto the timeline it has already emitted; the filter then keeps its converged state. Discriminating on step size alone cannot: a restart from a high counter value is indistinguishable from an overflow, and one from a low value looks like no anomaly at all.

3.5 Memory and timing design

Everything is statically sized. The dominating consumer in `ins_t` is the history ring buffer (30 entries $\times$ UDU factors); with the default compile-time maximum of 18 states the struct is ≈ 50 KiB, with `INS_UNKNOWN_MAX=15` ≈ 38 KiB. Per-epoch loops are bounded by compile-time constants or the runtime state count $n \leq \text{INS_UNKNOWN_MAX}$. Slowly varying Earth quantities (curvature radii, $\mathbf{R}_n^e$, gravity) are cached and refreshed only every 100 m of travel. The per-step latitude/longitude/height bookkeeping then costs some double multiply-adds (§5.3).

3.6 Sensor noise defaults

Any noise/tuning field left at zero is replaced by a conservative consumer-MEMS default (MPU6050/BMI160 class), so an uncharacterised sensor yields a stable (slightly cautious) filter instead of an overconfident one, but is not optimally tuned. The *gyro* defaults (ARW density, bias random walk) are shared between `ins` and `ahrs` via `sensor_defaults.h`: in the suite both filters observe the same physical gyro, so their fallback must agree. The *accelerometer* defaults are *not* shared – each filter fuses a different physical quantity: `ins` integrates the raw specific force (pure VRW density, $\approx 350 \mu\text{g}/\sqrt{\text{Hz}}$), `ars/ahrs` fuses a discrete leveling correction whose noise doubles as a manoeuvre-rejection margin (0.25 m s^{-2}), `baro_alt` propagates an attitude-projected vertical acceleration whose noise also absorbs roll/pitch projection error ($0.025 \text{ m/s}^2/\sqrt{\text{Hz}}$, $\approx 7 \times$ `ins`’s density – empirically not interchangeable).

4 Estimation backend: UDU factorisation

All filters¹ keep the covariance factorised as

$$\mathbf{P} = \mathbf{U} \text{diag}(\mathbf{d}) \mathbf{U}^\top, \quad (1)$$

with $\mathbf{U}$ unit upper triangular. In single precision a full $\mathbf{P}$ update loses symmetry and positive definiteness quickly; the factorised form is roughly equivalent to doubling the working precision and guarantees $\mathbf{P} \succeq 0$ by construction [3, 4, 7].

Prediction (Thornton). The time update

$$\mathbf{P}^- = \Phi \mathbf{P}^+ \Phi^\top + \mathbf{G} \text{diag}(\mathbf{Q}) \mathbf{G}^\top \quad (2)$$

is performed directly on the factors with Thornton’s modified weighted Gram–Schmidt orthogonalisation (`kalman_udu_predict`). Process noise enters in *noise-input form*: $\mathbf{G}$ maps physical white noise sources (sensor noise, bias random walks, per-state extra noise) onto the error states and $\mathbf{Q}$ holds the corresponding variances.

Measurement update (Bierman). Every fusion step assumes a linear(ised) measurement model

$$\mathbf{z} = \mathbf{H} \mathbf{x} + \mathbf{v}, \quad \mathbf{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{R}), \quad (3)$$

relating the state $\mathbf{x}$ to the measurement $\mathbf{z}$ through the observation matrix $\mathbf{H}$ and zero-mean noise $\mathbf{v}$ of covariance $\mathbf{R}$; for the error-state filters (`ins`, `ahrs`) $\mathbf{x}$ in (3) is the error state $\delta\mathbf{x}$, see (6) below. Measurements are processed as sequential scalar rows (Bierman rank-one updates, `kalman_udu`). For one scalar row with measurement vector $\mathbf{h}$ (a row of $\mathbf{H}$), residual z and measurement variance R , the classical Kalman fusion equations are

$$\mathbf{k} = \frac{\mathbf{P} \mathbf{h}}{\mathbf{h}^\top \mathbf{P} \mathbf{h} + R}, \quad \delta\mathbf{x} \leftarrow \delta\mathbf{x} + \mathbf{k} z, \quad \mathbf{P} \leftarrow (\mathbf{I} - \mathbf{k} \mathbf{h}^\top) \mathbf{P}, \quad (4)$$

¹Except the 1-state local-height/GNSS offset filter, where the covariance is a plain scalar variance and the Kalman equations are written out directly.

with $\mathbf{k}$ the Kalman gain. `kalman_udu` never forms (4) explicitly: it produces the same $\mathbf{k}$ and $\delta\mathbf{x}$ update while propagating the covariance change directly through the $\mathbf{U}$, $\mathbf{d}$ factors (Bierman’s rank-one algorithm), which avoids the numerically fragile $\mathbf{P} \leftarrow (\mathbf{I} - \mathbf{k}\mathbf{h}^\top)\mathbf{P}$ subtraction. Rows with correlated noise are decorrelated first: if the supplied $m \times m$ measurement covariance $\mathbf{R}$ has off-diagonal entries, $\mathbf{z}$, $\mathbf{H}$ and $\mathbf{R}$ are whitened with the Cholesky factor of $\mathbf{R}$ (`decorrelate()`), after which the scalar updates see independent unit-variance measurements.

Robust update. Each scalar row is screened with a chi-square test on its normalised innovation before it is applied [5]: with innovation z , innovation variance $S = \mathbf{h}^\top \mathbf{P} \mathbf{h} + R$ and threshold χ_{th}^2 ,

$$\frac{z^2}{S} > \chi_{\text{th}}^2 \implies \text{either skip the row, or inflate } R \text{ such that } \frac{z^2}{\mathbf{h}^\top \mathbf{P} \mathbf{h} + R'} = \chi_{\text{th}}^2. \quad (5)$$

The choice between *skip* and *downweight* (inflate) is the per-measurement-type policy of §3.4. A downweighted gross outlier receives a near-zero gain, but the measurement keeps contributing information, so a persistent offset is eventually followed instead of deadlocking the filter.

Error-state pattern. `ins` and `ahrs` are *error-state* (indirect) filters: the nominal state (quaternion, position, ...) is propagated by nonlinear mechanisation equations, while the Kalman filter estimates the small error $\delta\mathbf{x}$ with linear(ised) dynamics. The error state is implicitly zero after every update, so the Thornton step propagates only the covariance, and each fusion computes a correction $\delta\mathbf{x}$ that is immediately absorbed into the nominal state. The sign convention throughout the library is

$$\delta\mathbf{x} = \mathbf{x}_{\text{nominal}} - \mathbf{x}_{\text{true}}, \quad \mathbf{x}_{\text{nominal}} \leftarrow \mathbf{x}_{\text{nominal}} - \delta\mathbf{x}. \quad (6)$$

Residuals are formed as $z = h(\mathbf{x}_{\text{nominal}}) - z_{\text{measured}}$, which makes $z = \mathbf{H} \delta\mathbf{x} + v$ directly.

5 INS: the navigation ESKF

5.1 State definition

Nominal (total) state. Float-only, in the local n-frame:

Symbol	Field	Frame	Unit
$\mathbf{p}$	<code>pos_local</code>	NED, relative to origin	m
$\mathbf{v}$	<code>vel_ned</code>	NED	m s^{-1}
$\mathbf{q}_b^n$	<code>qbn</code>	body→NED	—
$\mathbf{b}_a$	<code>acc_bias</code>	body	m s^{-2}
$\mathbf{b}_g$	<code>gyr_bias</code>	body	rad s^{-1}
$\mathbf{b}_m$	<code>mag_bias</code>	body (for 18-state mode)	μT

The absolute position is captured in double precision, geodetically in both places: the n-frame origin as $(\varphi, \lambda, h)_{\text{origin}}$, and the current latitude/longitude/height (φ, λ, h) tracked incrementally from it (§5.3).

Error state. 15 elements, extended to 18 when magnetometer hard-iron bias estimation is enabled per instance at init:

$$\delta\mathbf{x} = \begin{bmatrix} \delta\mathbf{p}^\top & \delta\mathbf{v}^\top & \psi^\top & \delta\mathbf{b}_a^\top & \delta\mathbf{b}_g^\top & [\delta\mathbf{b}_m^\top] \end{bmatrix}^\top, \quad (7)$$

with $\boldsymbol{\psi}$ the small-angle attitude error in the n-frame (*psi-angle model*):

$$\mathbf{R}_b^n|_{\text{nominal}} = \left(\mathbf{I} + [\boldsymbol{\psi}]_{\times} \right) \mathbf{R}_b^n|_{\text{true}}. \quad (8)$$

A correction is applied as a left (n-frame side) quaternion multiplication $\mathbf{q} \leftarrow \mathbf{q}(-\boldsymbol{\psi}) \otimes \mathbf{q}$ (`ins_quat_small_angle_correction`), consistent with (6).

The 15/18 selection follows a *runtime-n* pattern: all arrays are dimensioned to the compile-time maximum `INS_UNKNOWN_MAX` (default 18), the active size n is fixed per instance at init. Memory-constrained integrations could override the maximum to 15 and reclaim the storage, `ins_init` then rejects the option instead of corrupting memory.

5.2 Strapdown mechanisation

Each IMU epoch propagates the nominal state over Δt (`ins_strapdown`). Earth rotation and transport rate are compensated:

$$\boldsymbol{\omega}_{in}^n = \boldsymbol{\omega}_{ie}^n + \boldsymbol{\omega}_{en}^n, \quad \boldsymbol{\omega}_{ie}^n = \omega_E \begin{bmatrix} \cos \varphi \\ 0 \\ -\sin \varphi \end{bmatrix}, \quad \boldsymbol{\omega}_{en}^n = \begin{bmatrix} v_E/(R_N + h) \\ -v_N/(R_M + h) \\ -v_E \tan \varphi/(R_N + h) \end{bmatrix}, \quad (9)$$

with ω_E the WGS84 Earth rate and R_M, R_N the meridian and normal curvature radii. The body rate relative to the n-frame is

$$\boldsymbol{\omega}_{nb}^b = \underbrace{\tilde{\boldsymbol{\omega}}_{ib}^b - \mathbf{b}_g}_{\text{bias-corrected gyro}} - (\mathbf{R}_b^n)^\top \boldsymbol{\omega}_{in}^n. \quad (10)$$

Attitude. The quaternion is advanced by the equivalent rotation vector $\boldsymbol{\omega}_{nb}^b \Delta t$, implemented in `ins_quat_rotate` with a Taylor-series expansion for numerical robustness near zero rotation [1].

Velocity. With the bias-corrected specific force $\mathbf{f}^b = \tilde{\mathbf{f}}^b - \mathbf{b}_a$, the n-frame correction terms

$$\mathbf{a}_{\text{corr}} = \mathbf{g}^n - (2\boldsymbol{\omega}_{ie}^n + \boldsymbol{\omega}_{en}^n) \times \mathbf{v} \quad (11)$$

(gravity including centrifugal term, plus Coriolis), and a second-order rotation (coning-style) correction of the integrated specific force [2, 1]

$$\Delta \mathbf{u}^b = \mathbf{f}^b \Delta t + \frac{1}{2} \boldsymbol{\omega}_{nb}^b \times \mathbf{f}^b \Delta t^2, \quad (12)$$

the velocity update is

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \mathbf{R}_b^n|_k \Delta \mathbf{u}^b + \mathbf{a}_{\text{corr}} \Delta t. \quad (13)$$

Note that $\mathbf{R}_b^n$ is evaluated at epoch k (before the attitude update); the rotation correction term accounts for the attitude motion across the interval.

Position. Trapezoidal integration:

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \frac{1}{2} \Delta t (\mathbf{v}_k + \mathbf{v}_{k+1}). \quad (14)$$

5.3 Incremental absolute-position bookkeeping

The filter states are float in the local n-frame, the absolute (φ, λ, h) is maintained in double precision from the *applied* n-frame deltas:

$$\Delta\varphi = \frac{\Delta p_N}{R_M + h}, \quad \Delta\lambda = \frac{\Delta p_E}{(R_N + h) \cos \varphi}, \quad \Delta h = -\Delta p_D. \quad (15)$$

The curvature terms (and gravity) change only by $\sim d/R_{\text{earth}}$ per metre of travel and are cached, refreshed after every 100 m of accumulated travel. $\mathbf{R}_n^e$ is not among them: no epoch-path computation uses it, so it is built on demand in `ins_get_velocity_ecef()` rather than refreshed inside the worst-case epoch.

5.4 Covariance prediction

The prediction is throttled: it runs when at least `kalman_update_dt_sec` has elapsed (default order 50 ms to 100 ms), not at IMU rate – the error dynamics are slow, and the strapdown already propagates the nominal state per sample. The discrete transition matrix over the elapsed Δt is [1, eq. 10.1]

$$\Phi = \begin{bmatrix} \mathbf{I} & \mathbf{I}\Delta t & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} & -[\mathbf{f}^n]_{\times} \Delta t & -\mathbf{R}_b^n \Delta t & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I} & \mathbf{0} & -\mathbf{R}_b^n \Delta t \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} \end{bmatrix}, \quad \mathbf{f}^n = \mathbf{R}_b^n \mathbf{f}^b, \quad (16)$$

where $\mathbf{f}^b$ is the latest bias-corrected specific force. The $-[\mathbf{f}^n]_{\times}$ block is the velocity/attitude coupling under the sign convention (6)(8): $\delta\dot{\boldsymbol{\psi}} = [\boldsymbol{\psi}]_{\times} \mathbf{f}^n = -[\mathbf{f}^n]_{\times} \boldsymbol{\psi}$. In 18-state mode the magnetometer bias block is appended as identity (random walk).

Process noise enters (2) through

$$\mathbf{G} = \left[\begin{array}{cccc|c} \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I}_n \\ \mathbf{R}_b^n & \mathbf{0} & \mathbf{0} & \mathbf{0} & \\ \mathbf{0} & -\mathbf{R}_b^n & \mathbf{0} & \mathbf{0} & \\ \mathbf{0} & \mathbf{0} & \mathbf{I} & \mathbf{0} & \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} & \end{array} \right], \quad \mathbf{Q} = \Delta t \cdot \begin{bmatrix} \text{PSD}_{\text{acc}} \\ \text{PSD}_{\text{gyr}} \\ \sigma_{b_a\text{-RW}}^2 \\ \sigma_{b_g\text{-RW}}^2 \\ \mathbf{q}_{xx} \end{bmatrix}, \quad (17)$$

i.e. twelve IMU noise inputs (accelerometer and gyro white noise rotated into the n-frame, both bias random walks) plus n per-state extra-noise inputs $\mathbf{q}_{xx}$ (position/velocity/attitude stabilisation noise and, in 18-state mode, the magnetometer-bias random walk). The measurement-supplied per-axis PSDs are used when present, axes left at zero fall back to the conservative MEMS defaults (§3.6) rather than injecting zero process noise.

5.5 Measurement models

All fusions follow the pattern of §4: build the residual $\mathbf{z} = h(\text{nominal}) - \mathbf{z}_{\text{meas}}$ and the Jacobian $\mathbf{H}$, run the robust Bierman update, subtract the estimated $\delta\mathbf{x}$ from the nominal state.

5.5.1 GNSS position

The measurement arrives as geodetic (φ, λ, h) in double precision, the form receivers report, with a full 3×3 NED covariance. No coordinate conversion runs on the epoch path: the fix is differenced against the filter’s anchor at the measurement’s time of validity directly, and the difference is mapped to metres with the curvature radii. A source that is natively ECEF (RTK,

UBX-NAV-HPPOSECEF) converts once in the caller, outside the filter's worst case. With the GNSS antenna lever arm $\mathbf{l}^b$:

$$\mathbf{z}_p = \underbrace{\mathcal{M}((\varphi, \lambda, h)_{\text{tov}} - (\varphi, \lambda, h)_{\text{meas}})}_{\text{NED difference}} + \mathbf{R}_b^n|_{\text{tov}} \mathbf{l}^b, \quad (18)$$

$$\mathbf{H}_p = \begin{bmatrix} \mathbf{I}_3 & \mathbf{0} & -[\mathbf{l}^n]_{\times} & \mathbf{0} & \mathbf{0} \end{bmatrix}, \quad \mathbf{l}^n = \mathbf{R}_b^n|_{\text{tov}} \mathbf{l}^b, \quad (19)$$

following [1, eq. 8.62]. The $-[\mathbf{l}^n]_{\times}$ block makes a lever-arm-induced residual observable in the attitude states: without it, the filter would attribute the residual purely to a position error and could not extract heading/tilt information from an offset antenna.

Quality gates reject measurements whose reported standard deviations exceed configured horizontal/vertical limits before fusion (counted in the diagnostics). When `ins` uses the barometric height source (§5.5.2), only the horizontal (N, E) rows of (19) are fused – the vertical row is dropped, not downweighted: the two height sources are not simultaneous alternatives for the same epoch, the selection is fixed once at bootstrap. The position gates then grade the fix on its horizontal accuracy alone, both here and at the 3D exit gate (§5.9): a fix whose vertical row is discarded before fusion must not lose the horizontal solution its aiding, nor drop the whole 3D solution, over an accuracy it no longer contributes – a good HDOP with a poor VDOP is a typical satellite geometry. The velocity gates are unaffected on all three axes, as is the *entry* gate's vertical position limit: the entry gate runs during a cold bootstrap, before any height source is latched, and that one fix defines the origin, so its vertical accuracy is the accuracy of the absolute vertical datum whatever runs on top of it afterwards.

5.5.2 Barometric height (ins-internal)

If barometer data was present when INS started (§5.7), the barometric altitude is fused directly as a scalar measurement of the vertical position state, on the same datum anchor h_0 established at bootstrap (§5.10, §7.1) and using the same ISA conversion as `baro_alt` (46):

$$z_h = -p_z - (h_{\text{ISA}}(p) - h_0), \quad \mathbf{H}_h = \begin{bmatrix} -\mathbf{e}_3^T & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix}, \quad (20)$$

with $\mathbf{e}_3 = (0, 0, 1)^T$ picking the down component of $\mathbf{p}$ ($h = -p_z$, §7.1). Fused through the same robust UDU update as every other measurement (§4), outliers downweighted like the barometer's own fusion in `baro_alt` (§7.2) – a persistent absolute reference, not a high-rate local one (§3.4). This replaces, not supplements, the vertical row of the GNSS position measurement (19) for the lifetime of the filter instance: the choice of height source is made once, at bootstrap, from whether a barometer was observed (§5.7), not re-evaluated per epoch. GNSS velocity (§5.5.3), including its vertical component, is fused unconditionally either way (§10).

Like every other fusion this runs on live epochs only: while the coasting window is expired the filter is inert (§5.9). The barometer keeps being cached across such an outage, and the height is picked back up from it at the re-acquisition, which is what keeps the vertical solution continuous across a passage the frozen state knew nothing about.

5.5.3 GNSS velocity

NED velocity with full covariance, the lever-arm velocity is compensated and its attitude coupling included [1, eq. 8.74]:

$$\mathbf{z}_v = \mathbf{v}_{\text{tov}} + \mathbf{R}_b^n|_{\text{tov}} (\boldsymbol{\omega}_{nb}^b \times \mathbf{l}^b) - \mathbf{v}_{\text{meas}}, \quad \mathbf{H}_v = \begin{bmatrix} \mathbf{0} & \mathbf{I}_3 & -[\mathbf{R}_b^n(\boldsymbol{\omega}^b \times \mathbf{l}^b)]_{\times} & \mathbf{0} & \mathbf{0} \end{bmatrix}. \quad (21)$$

(Using $\boldsymbol{\omega}_{ib}^b$ instead of $\boldsymbol{\omega}_{nb}^b$ neglects the Earth/transport rate crossed with the GNSS antenna lever arm, $\sim 0.1 \text{ mm s}^{-1}$ for a 1 m arm – negligible against GNSS velocity noise.) When position

and velocity are fused together, an optional position/velocity cross-covariance block completes the full 6×6 measurement covariance, correlated blocks are whitened before the scalar updates (§4).

5.5.4 GNSS covariance conditioning

A receiver’s reported covariance is not fused as it stands. It passes a fixed pipeline whose knobs live under `gnss:` in `config.yaml` (§11), applied per epoch to the position and velocity blocks before either reaches (19):

$$\text{envelope} \rightarrow \text{scale} \rightarrow \text{height scale} \rightarrow \text{floor} \rightarrow \text{cap} \rightarrow \text{manoeuvre term}. \quad (22)$$

Envelope. A rise in the reported 1σ takes effect at once, a fall only with a first-order decay of `acc_envelope_tau_sec`: $e \leftarrow \max(\sigma_{\text{rep}}, e(1 - \alpha))$, $\alpha = \text{clamp}(\Delta t/\tau, 0, 1)$, kept per block and axis group. A receiver’s accuracy figure returns to nominal well before a re-acquired fix has settled.

Scale and height scale. $\mathbf{Q} \leftarrow s^2 \mathbf{Q}$, plus an extra downweight of the position Down axis applied as the correlation-preserving congruence $\text{diag}(1, 1, s_h) \mathbf{Q} \text{diag}(1, 1, s_h)$.

Floor and cap. Per-axis clamps of the diagonal (horizontal/vertical separately), so a floored or capped axis picks up (or loses) independent variance on that axis alone. The cap is what makes the loose fusion gate safe: `gnss_max_*` defaults to the caps, so a degraded fix is downweighted rather than discarded while the unbounded reported accuracy stays out of the 32-bit hot path.

Manoeuvre term. Extra independent variance on the velocity diagonal, $\sigma_{\text{extra}} = k\|\mathbf{a}\|$, from the windowed n -frame acceleration of the *antenna* – the body’s plus the centripetal term $\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{l})$ of the lever arm. A Doppler velocity averages over the receiver’s measurement interval T while the state is instantaneous, so under acceleration the two differ by about $aT/2$, an error the reported accuracy cannot describe because it belongs to the platform rather than to the signal.

All of it weights the *fusion* only. Every fix-quality gate (§5.9) keeps grading the stddev the receiver reported, so downweighting a noisy receiver can never turn into refusing to use it.

5.5.5 Absolute speed (odometry, OBD-II)

A scalar ground speed – an OBD-II vehicle speed, a wheel-odometry pulse rate, a Doppler log – is the magnitude of the velocity with no direction attached:

$$h(\mathbf{x}) = \|\mathbf{v}^n\|, \quad z_s = \|\mathbf{v}_{\text{tov}}\| - z_{\text{meas}}, \quad \mathbf{H}_s = \begin{bmatrix} \mathbf{0} & \hat{\mathbf{v}}^\top & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix}, \quad \hat{\mathbf{v}} = \frac{\mathbf{v}^n}{\|\mathbf{v}^n\|}. \quad (23)$$

The nonlinearity costs nothing in an error-state filter: the residual is formed from the nonlinear $h(\cdot)$ evaluated on the *nominal* state and $\mathbf{H}$ is only its Jacobian with respect to the error state, exactly as for every other reference here.

$\hat{\mathbf{v}}^\top$ picks out the component of the velocity error *along the current direction of travel* and nothing else; cross-track error is invisible to a quantity that carries no direction. The along-track component is nevertheless the one that runs away first when GNSS stops, since attitude and accelerometer-bias errors leak into velocity through the strapdown and it is the integrated velocity error that becomes position error. One scalar per second therefore buys most of what a GNSS-denied stretch needs, without the state cloning a delta-position (wheel-tick) formulation would require.

Two properties of the sensor class shape the implementation. First, $\hat{\mathbf{v}}$ is undefined at $\mathbf{v}^n = \mathbf{0}$ and numerically meaningless just above it, so the fusion is gated on a minimum *filtered* speed:

what has to be well conditioned is the Jacobian, and that is built from the state, not from the reading. A genuinely stopped platform is the automatic ZUPT's job (§5.8).

Second, the error budget has two parts that scale differently, and lumping them together would bias the filter. An OBD-II PID 0x0D value is quantised to 1 km h^{-1} , a $0.080 \text{ m s}^{-1} 1\sigma$ that does not grow with speed. The scale error of the vehicle's speed signal does: rolling radius, tyre wear and the scaling the ECU applies are a few percent, which at the library's default 3 % is 0.83 m s^{-1} at 100 km h^{-1} – ten times the quantisation noise and *systematic*. Its sign is not predictable from the type-approval margin that keeps an indicated speed from ever falling below the true one: that constrains the dashboard, whereas PID 0x0D is the ECU's own value and a different number from the one the cluster shows. A constant scale factor removes the calibrated part and a relative variance prices what is left:

$$z_{\text{meas}} = s_{\text{scale}} z_{\text{raw}}, \quad R = \sigma_{\text{sample}}^2 + (k_{\text{rel}} z_{\text{meas}})^2. \quad (24)$$

No scale-factor state is estimated: this stays a measurement-boundary feature and leaves the runtime-selected state size untouched. `tools/inslib_speed_scale.py` fits s_{scale} against GNSS speed and prints the `config.yaml` value.

The reading is unsigned by contract (an OBD-II speed PID cannot express reverse), so a negative value is treated as a unit or sign error and dropped rather than taken as its magnitude. Latency is the norm on this input – a Bluetooth OBD-II round trip is tens of milliseconds – and is handled by the same history anchoring GNSS uses (§5.6).

5.5.6 Ranges to known anchors

A range measurement is the distance between the platform's ranging antenna and an anchor at a known position. Up to `INS_RANGE_MAX` entries per epoch go into `ins_measurements_t.range[]`, each with the anchor position in ECEF (`double`), the measured range, its 1σ , its own delay and an anchor id for diagnostics. One lever arm (`range_leverarm_b`) applies to all entries. Each entry is one scalar row:

$$h(\mathbf{x}) = \|\mathbf{a}^e - \mathbf{p}_{\text{ant}}^e\|, \quad z_r = h(\mathbf{x}_{\text{tov}}) - z_{\text{meas}}, \quad \mathbf{H}_r = \begin{bmatrix} -\mathbf{u}^\top & \mathbf{0} & \mathbf{u}^\top [\mathbf{l}^n]_\times & \mathbf{0} & \mathbf{0} \end{bmatrix}, \quad (25)$$

with $\mathbf{u}$ the unit line-of-sight vector from the antenna to the anchor in the n-frame and $\mathbf{l}^n$ the lever arm. $\mathbf{H}_r$ is the local-position Jacobian projected onto $-\mathbf{u}$, so the lever arm couples the range into the attitude.

The fusion is split in two layers. A geometry layer turns an entry into $\mathbf{u}$, z_r and a variance, and a fusion kernel only sees those three.

Geometry without per-epoch trigonometry. The filter's absolute position is geodetic, the anchors are ECEF. Converting every epoch would cost double precision trigonometry, which is slow on single-precision embedded targets. The antenna position is therefore formed from a cached reference point (geodetic position, its exact ECEF position, its NED-to-ECEF rotation) plus the n-frame offset from it, i.e. with double additions only. The reference is recomputed exactly once the position moves farther than `INS_RANGE_REF_RADIUS_M` from it. The error of the approximation is about $s^2/R_\oplus$ at a distance s from the reference, millimetres inside the radius. The range is rotation invariant, so the reference's rotation instead of the current one only tilts $\mathbf{u}$ by $s/R_\oplus$. The anchor minus antenna difference is formed in double, only the result goes to single precision, which resolves terrestrial anchors to millimetres. Satellite distances will need the norm in double as well, together with the pseudorange geometry layer.

Barometric height. While the filter runs on the barometric height source (§5.5.2), ranges correct North/East only by default: the Down component of $\mathbf{u}$ is removed from $\mathbf{H}_r$ and the

height uncertainty is considered instead of corrected:

$$R = \sigma_{\text{meas}}^2 + u_D^2 P_{DD}. \quad (26)$$

Zeroing the Down column alone would leave a height error in z_r and leak it into North/East. The barometric datum and the anchors' ellipsoid heights disagree, and a multipath-lengthened range over a steep line of sight would push a height the barometer determines better. GNSS position fusion is restricted to the horizontal under barometric height for the same reason. `opt.range_height_with_baro` switches the height correction back on, for installations whose anchors sit well above and below the platform. Without the barometric height source ranges always correct all three axes.

Scope and coasting. Ranges are fused on a live filter only. They neither initialize nor re-acquire it. Whether an epoch's ranges count as position aiding for the coasting window (Figure 2), like a GNSS fix, is decided by the covariance after the fusion: they count if the horizontal position 1σ in its worst direction, the square root of the larger eigenvalue of the North/East block, stays at or below `opt.range_aiding_max_hpos_stddev_m`. A single range bounds the position along its line of sight only, so one anchor keeps the filter alive only until the across-track uncertainty has grown past the limit, while anchors with a usable geometry keep it ready through a GNSS outage and also keep it from the GNSS quality-loss exit. `nav_suite` stays in FULL mode as long as they count. The criterion is only as good as the stated range 1σ : multipath is a positive bias, and too small a 1σ keeps the filter ready on a biased solution. The anchors are absolute, so a `nav_suite` local-frame offset does not apply to them.

5.5.7 Position decimation against an unknown correlation

That optional cross-covariance block is the exception, not the rule. A receiver computes position and velocity in one internally coupled solution, but no common output message carries the correlation between them: `gnss_Q11_pos_vel_ned` is filled only by the few receivers that report it and is zero otherwise. Fusing both blocks every epoch as if they were independent therefore counts one solution twice. The covariance drops below what the measurements support, which in turn shrinks the Kalman gain of every other aiding source and pushes the difference into the accelerometer-bias and attitude states that are free to absorb it.

`gnss.pos_decimation` answers this by spending only one block per epoch. With the factor N , every N -th epoch that offers a usable position *and* a usable velocity fuses the position alone, the other $N - 1$ fuse the velocity alone; $N \leq 1$ restores the combined fuse. An epoch offering only one of the two is never touched – the decimation removes information, it never adds any.

Decimation is deliberately the blunt instrument: it assumes nothing about a correlation that cannot be observed, unlike an assumed correlation coefficient, which is unverifiable against a receiver that reports none and drives $\mathbf{R}$ towards singularity when guessed too high. It also attacks the second, larger correlation in the same data: a receiver's position error is dominated by multipath, residual ionospheric delay and its own smoothing, all correlated over minutes, while the Doppler-derived velocity is comparatively white. The textbook cure for that time-correlated part is a Gauss-Markov position-bias state per axis, which would allow full rate for both – but that is a change to the state vector, not what this option does.

Two cases suspend the decimation. While the coasting window is expired (§5.9) the position is the entire point of the fix, so it is always fused and re-anchors the filter. And a position-aiding gap longer than the fix-stream continuity bound restarts the cycle at a position, because after a gap the dead-reckoned position is the stale quantity. An epoch whose position is withheld still counts as position aiding for the coasting window: the fix carried a usable position and the filter declined to spend it, which is not the same as not having one.

5.5.8 Local NED position

Indoor references (e.g. Lighthouse/SteamVR, UWB, motion capture, total station) measure `pos_local` directly in the filter’s n-frame – same residual and Jacobian structure as (19), without any coordinate conversion. Frame alignment (origin offset + rotation into NED) is the caller’s responsibility. Outliers are *skipped*, not downweighted (§3.4).

5.5.9 Absolute yaw

A heading reference (dual-antenna GNSS, mocap pose, gyro compass) is fused as a scalar residual on the yaw component of the attitude error:

$$z_\psi = \text{wrap}_{\pm\pi}(\hat{\psi}_{\text{tov}} - \psi_{\text{meas}}), \quad \mathbf{H}_\psi = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{e}_3^\top & \mathbf{0} & \mathbf{0} \end{bmatrix}, \quad (27)$$

since to first order a heading residual equals the yaw component of the n-frame attitude error. Fusion is skipped near gimbal lock ($|\sin \theta| > 0.99$), where yaw is ill-defined.

Dual-antenna heading. A moving base receiver pair (u-blox NAV-RELPOSNED `relPosHeading`) measures the azimuth h of the vector between its two antennas, not the vehicle yaw. With that baseline $\mathbf{b}^b$ fixed in the body frame, the level-frame baseline $\mathbf{c} = \mathbf{R}_y(\theta) \mathbf{R}_x(\phi) \mathbf{b}^b$ has azimuth $\psi + \text{atan2}(c_y, c_x)$, so

$$\psi_{\text{meas}} = \text{wrap}_{\pm\pi}(h - \text{atan2}(c_y, c_x)) \quad (28)$$

is exact for any mounting and tilt (`ins_yaw_from_baseline_heading()`, REQ-NAV-087). For a baseline along the body x axis $c_y = 0$ and the azimuth is the yaw at any roll and pitch. Any other mounting couples the attitude in: a baseline across the vehicle reads 3.6 deg off at 20 deg roll and 10 deg pitch if fed as yaw directly. The function refuses a baseline whose horizontal projection is shorter than a tenth of its length, where antenna phase noise dominates the azimuth. The firmware glue and both replay harnesses call it with the attitude the suite holds at the time, level while it holds none, and pass the receiver’s heading delay as `yaw_delay_ms`. With no magnetometer and no motion this is also what gives auto-init its yaw.

Automotive mode (course-over-ground yaw). Opt-in (`automotive_mode`), this reuses the same scalar update with $\psi_{\text{meas}} = \text{atan2}(v_E, v_N)$ from the GNSS velocity vector, above a minimum ground speed (`automotive_min_speed_mps`, default 2 m s^{-1} – below it course over ground is not well defined). It assumes a non-holonomic vehicle that travels in the direction it points – reasonable for a car, wrong for a glider or a boat drifting sideways: in coordinated flight the airframe’s heading aligns with the *airspeed* vector, not the *ground-track* vector GNSS measures, so a crosswind adds a wind-drift (crab) angle independent of speed – raising the minimum speed does not fix this. The measurement standard deviation is the velocity-noise-over-speed heading uncertainty, floored (`automotive_min_yaw_stddev`, default 5°) so it never claims more precision than that assumption warrants. Raising this floor (e.g. to a conservative crab-angle estimate such as 15°) lets the fusion still act as a weak long-term anchor against free-yaw drift on a platform where the assumption itself is looser than a car’s, without asserting automotive-grade heading precision. Outliers (e.g. reversing, $\sim 180^\circ$ residual) are chi-square-downweighted like any absolute yaw reference, never dropped. An alternative to magnetometer heading aiding when none is available.

5.5.10 Magnetometer (yaw-only vector fusion)

The magnetometer is fused as a 3D vector measurement of the body-frame field with model $\mathbf{m}^b = (\mathbf{R}_b^n)^\top \mathbf{m}^n + \mathbf{b}_m$:

$$\mathbf{z}_m = (\mathbf{R}_b^n)^\top \mathbf{m}^n + \mathbf{b}_m - \tilde{\mathbf{m}}^b. \quad (29)$$

The full attitude Jacobian of this model would be $(\mathbf{R}_b^n)^\top [\mathbf{m}^n]_\times$ on the ψ block. *Deliberately, only the yaw column is kept:*

$$\mathbf{H}_m[:, \psi_z] = (\mathbf{R}_b^n)^\top (\mathbf{m}^n \times \mathbf{e}_D), \quad \mathbf{H}_m[:, \psi_x] = \mathbf{H}_m[:, \psi_y] = \mathbf{0}, \quad \frac{\partial \mathbf{z}_m}{\partial \delta \mathbf{b}_m} = \mathbf{I}_3. \quad (30)$$

The zeroed roll/pitch columns ensure that magnetic dip and hard-iron / deviation disturbances can never tilt the attitude estimate – leveling information comes exclusively from the accelerometer. The vector measurement can only rotate the estimate about the vertical, residual components that a yaw rotation cannot explain (magnitude or dip mismatch) meet zero entries and produce no state update [1, eq. 10.7].

Hard-iron bias observability (18-state mode). The bias term is constant in the *body* frame while the field term $(\mathbf{R}_b^n)^\top \mathbf{m}^n$ rotates with attitude. This makes $\mathbf{b}_m$ separable from the yaw error – but only under attitude changes. Expect convergence during/after rotations, this is why the bias is estimated *inside* the filter, where the yaw/bias cross-covariance is maintained, instead of in an external calibration estimator.

Tuning the hard-iron states. Two parameters govern them: `mag_bias_init_stddev_ut` (10 μT by default), the prior on how much hard iron the filter should be prepared to find, and `mag_bias_pred_stddev_ut_sqrts` (0.129 $\mu\text{T}/\sqrt{\text{s}}$ by default, which is 1 μT per minute), the random walk. Neither is right everywhere. The initial sigma is the stronger lever: a recording whose magnetometer was calibrated where it sits wants a tight prior, while a unit just installed in an unknown vehicle does not, since 10 μT to 20 μT of vehicle hard iron is ordinary. The random walk is better read as how long the estimate keeps listening after it has settled than as a physical drift rate: hard iron steps when ferrous mass moves rather than wandering continuously, and what the number decides is whether such a step is still followed. Against a per-axis sigma of a few μT fused at about 1 Hz the state settles near $(q^2 \Delta t R)^{1/4}$ and admits $q\sqrt{T}$ of drift over a time T ; too small a value converges to a confidence it then cannot be talked out of. Both are exposed to the replay harness as `mag.bias_init_ut` and `mag.bias_rw_ut_sqrts`.

Field-strength disturbance gate. Once a position is known (§9), the measured magnitude is compared against the WMM total field B_{WMM} . With relative deviation $\epsilon = \|\hat{\mathbf{r}}\mathbf{m}\| - B_{\text{WMM}}/B_{\text{WMM}}$ and tolerance τ (default 0.30), a sample outside the band has its per-axis noise inflated by $(\epsilon/\tau)^2$ – downweighted, not dropped, per the persistent-reference policy. The gate is defense-in-depth over the vector chi-square test (it bites when the measurement noise is set loose) and can be disabled for uncalibrated magnetometers. Fusion requires per-axis variances (no default) and is rate-limited: it is skipped when the horizontal reference field is unusable, either because the reference vector itself is degenerate or because the position lies inside a dip pole exclusion zone (§9.2).

Weight and rate limit: a long-term anchor, not a yaw sensor. Since only the yaw column survives, the length of that column is the horizontal field $\|\mathbf{m}_h^n\|$, so a per-axis noise σ_m acts as a heading measurement of roughly

$$\sigma_\psi \approx \frac{\sigma_m}{\|\mathbf{m}_h^n\|}. \quad (31)$$

At mid-latitudes $\|\mathbf{m}_h^n\| \approx 20 \mu\text{T}$, so even a sensor-grade $\sigma_m = 3 \mu\text{T}$ is already about 8.6° of heading per sample – and at a typical 10 Hz output rate the filter accumulates that down to under a degree within seconds. A MEMS gyro of $0.02^\circ/\text{s}/\sqrt{\text{Hz}}$ drifts far less than that over the same seconds, so fusing at the sensor rate buys no heading accuracy. What it does buy is

exposure: motor currents and ferrous structure bend the field by far more than σ_m , and those disturbances are strongly time-correlated while the update treats every sample as independent evidence, so a high rate systematically over-counts a biased heading.

Both knobs therefore default weak, and both defaults live in the library rather than in each harness: a sample that arrives with no per-axis variance is fused with `INS_DEFAULT_MAG_STDDEV_UT` (`sensor_defaults.h`), a noise of the same order as the field itself, i.e. $\sigma_\psi \approx 1$ rad per sample, and the rate limit defaults to 1 Hz (`ins_options_t.magnetometer_min_delay_ms`, $0 \rightarrow$ default, negative $\rightarrow$ off). That combination still binds the heading over minutes, which is all the gyro needs, without letting a disturbed field pull it within seconds. Note that this is a defence against *noise and transients only*: a standing hard-iron offset is a *bias*, and where the magnetometer is the only absolute heading reference the steady-state error it causes is independent of σ_m – lowering the weight only stretches the time the filter takes to reach the same wrong answer. (It does shift the balance once a second heading source is present, e.g. GNSS course in automotive mode.) Bias is the calibration’s job (`mag: fixed_bias / estimate_bias`), not the weighting’s.

5.5.11 Zero-velocity and zero-rotation updates (ZUPT/ZARU)

On trigger (external flag or the automatic detector, §5.8):

$$\text{ZUPT: } \mathbf{z} = \hat{\mathbf{v}}, \quad \mathbf{H} = \begin{bmatrix} \mathbf{0} & \mathbf{I}_3 & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix}, \quad (32)$$

$$\text{ZARU: } \mathbf{z} = \hat{\mathbf{b}}_g + (\mathbf{R}_b^n)^\top \boldsymbol{\omega}_{in}^n - \bar{\boldsymbol{\omega}}_{\text{meas}}, \quad \mathbf{H} = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I}_3 \end{bmatrix}, \quad (33)$$

i.e. the zero-rotation update is a direct gyro-bias measurement, with the Earth + transport rate seen in the body frame compensated. Both are rate-limited to one fusion per Kalman period. $\bar{\boldsymbol{\omega}}_{\text{meas}}$ is the gyro *averaged over the current stillness run* rather than a single sample: a momentary sample of a vibrating-but-still platform (idling engine) would inject the vibration directly into the gyro bias states – with a tight σ_{ZARU} this corrupted the heading by up to 0.6°s^{-1} at every stop on real automotive data. The average is a running mean, not a median: bounded memory and $O(1)$ per sample (WCET rule).

5.6 Delayed measurements: the history buffer

GNSS (and optionally local position / yaw) measurements arrive with latency of up to 500 ms (`INS_MAX_DELAY_MS`). The filter keeps a ring buffer of recent states (nominal state, UDU factors, (φ, λ, h) , $\mathbf{R}_b^n$, $\boldsymbol{\omega}$; one entry every ≥ 20 ms, 30 entries). A delayed measurement’s residual is formed against the state at its time of validity, while the covariance update is applied to the *current* $\mathbf{U}, \mathbf{d}$. This is the standard approximation for short delays: the error state is nearly constant over a few hundred milliseconds ($\Phi \approx \mathbf{I}$ for the error dynamics), whereas the residual would be grossly wrong if formed against the already-moved-on current state. A measurement that cannot be anchored (no history entry within 50 ms of the requested time) is skipped and counted.

5.7 Auto-initialisation

When enabled, `ins_init` leaves the filter in a collecting state, `ins_update` buffers bias-corrected IMU samples (bounded buffer) until the first usable position fix (GNSS or local). At that point:

- **Roll/pitch** from accelerometer leveling over a static window – per-axis *medians* over the window (allowed here: init path, not the per-epoch path), gated on gyro magnitude and $|\|\mathbf{f}\| - g|$:

$$\phi = \text{atan2}(-f_y, -f_z), \quad \theta = \text{atan2}\left(f_x, \sqrt{f_y^2 + f_z^2}\right). \quad (34)$$

- **Yaw** by priority: external yaw measurement (with its variance), else tilt-compensated magnetometer heading against the reference field (with the configured initial yaw variance), else unknown – yaw stays 0 with a $\sigma_\psi = \pi$ prior, letting the running fusion pull it in (e.g. from lever-arm or velocity coupling).
- **Position/velocity** from the fix; with a local-position fix the caller’s geodetic anchor is kept as origin and the fix sets `pos_local`, so subsequent local measurements stay consistent. A GNSS fix locates the antenna: the origin is the fix, and the IMU starts at `pos_local` = $-\mathbf{R}_{b \rightarrow n} \mathbf{l}_b$, the lever arm rotated with the attitude just determined. The initial velocity is the fix’s minus $\mathbf{R}_{b \rightarrow n}(\boldsymbol{\omega} \times \mathbf{l}_b)$, the antenna’s own motion on a turning platform. With the yaw still unknown only the vertical components are applied, since a horizontal arm rotated by an arbitrary yaw can land further off than not moving it at all.
- **Height source** is latched once here, for a GNSS bootstrap only, from whether a barometer is *streaming* at this instant: the most recent plausible sample cached during the collecting window must exist *and* still be current (`INS_BARO_ANCHOR_MAX_AGE_SEC`). Unlike the GNSS entry gate (§5.9) the sample is not quality-graded – a pressure reading carries no reported covariance – but bare presence is not enough either, since the decision is never revisited and that same sample becomes the datum anchor. Streaming $\rightarrow$ barometric height (§5.5.2) with the anchor h_0 taken as `ins`’s own h_{ISA} of that sample at $h_{\text{init}} = 0$, matched to `baro_alt`’s independently-running height by the vertical-datum alignment right after this call returns (§3.3) rather than read from `baro_alt`. Absent or stale $\rightarrow$ the vertical row of the GNSS position (19) as before, with a warning for the stale case: a barometer that stopped during the collecting window is a fault, not a configuration.

Two ways out of the barometric source. A local-position bootstrap never selects it regardless of barometer presence, its own vertical accuracy being typically better than a barometer’s. And `baro_height_disable` suppresses it outright, for the same situation reached through the other interface – a GNSS-labelled input that is not satellite GNSS and already reports a better-than-barometric vertical accuracy, e.g. a precise indoor tracking system fed through `gnss_pos/gnss_vel` for convenience.

Not re-evaluated afterwards: a barometer that later fails is a plain aiding gap on that one measurement (§10), not a source change. Since that gap leaves the vertical channel without an absolute reference while every other health signal stays green, it is reported explicitly (`ins`: “no barometric height aiding for ...”, alongside the `n_baro_height_used` counter).

The initial yaw variance is independently configurable from roll/pitch (`init_stddev: yaw_init_stddev_rad_deg, ins_init_t.rpy_init_stddev_rad[2]`): leveling and heading come from physically different accuracy sources.

5.8 Automatic ZUPT/ZARU detection

A stillness detector (enabled by default) arms the ZUPT/ZARU fusions without an external trigger, tests on three independent criteria that must all hold:

Variance (primary statement). The per-axis sample *variance* of the raw (not bias-corrected) gyro and accelerometer over a short window, evaluated when the window covers a minimum duration and sample count, with the result latched until the next window completes. Variance rather than magnitude because a constant sensor bias shifts the sample mean but not its spread: a magnitude-only gate reads a biased but perfectly stationary IMU as moving, and that failure is self-locking (this detector is what would estimate the bias in the first place). Configurable (`auto_zupt_static_gyr_stddev_deg / auto_zupt_static_`

`acc_stddev_mps2`, $0 \rightarrow$ built-in default), raising them out of range effectively disables the criterion, which a platform whose standstill is inherently noisy needs (a station-keeping multicopter puts more spread on the IMU from its rotors than handheld motion does).

Magnitude bounds. $\|\boldsymbol{\omega}\| \leq \omega_{\max}$, $|\|\mathbf{f}\| - g| \leq a_{\max}$, applied to the *bias-corrected* sample. Retained alongside the variance criterion because variance alone cannot distinguish a constant rotation rate or centripetal acceleration (steady turntable or circular drive: near-zero spread, plainly not stationary) from a standstill – the two criteria cover different failure modes and neither subsumes the other.

Velocity gate. $\|\mathbf{v}_{\text{GNSS}}\| \leq v_{\max}$, using *exclusively* a recent, sufficiently precise external (GNSS) velocity observation. The observation must additionally report its own 1-sigma (per axis) at or below `auto_zupt_max_vel_stddev_mps`: a low reported speed with a large uncertainty is not evidence of standstill. Without a GNSS velocity meeting both the recency and precision requirement, this gate does not apply – it passes unconditionally and only the IMU criteria above decide. That is a design decision, so that the ZUPT updates are not hardwired to GNSS availability.

Triggering additionally requires a dwell time and is rate-limited. During the stillness run the raw gyro is accumulated for the ZARU average. All thresholds above, plus $v_{\max}$ (`auto_zupt_max_vel_mps`), are reachable from both replay harnesses’ `config.yaml` (`imu.auto_zupt_*`, §11), each $0 \rightarrow$ the built-in default.

The detector can also be switched on/off at runtime (`ins_set_auto_zupt_disable`), independent of the `auto_zupt_disable` option resolved at init – e.g. a multicopter briefly holding position mid-flight can guarantee no zero-velocity/zero-rotation update fires for exactly that window and resume automatic detection once grounded. Disabling clears the in-progress dwell timer and the latched variance verdict, so re-enabling always requires a fresh stillness run rather than firing off a stale timer. Airborne vehicles should make use of this.

The ARS and magnetometer-AHRS (§3.3) have no velocity state, so they run their own, independent fallback stillness detector against the same physical IMU: the variance and magnitude criteria above, minus the velocity gate `ins` can apply. It therefore also arms during genuine constant-velocity cruise – an accepted cost, since it is armed by default and is the only way to correct the otherwise permanently unobservable ARS/AHRS z-axis gyro bias when `ins` has no trigger to propagate (e.g. no aiding source at all).

Separate implementations, but *one* configuration: inside `nav_suite` the thresholds, dwell and pseudo-measurement stddevs of that fallback are not set on `ahrs_config_t` at all. `nav_suite_init()` fills them from the same `ins_options_t.auto_zupt_*` set described above, and `baro_alt`’s vertical zero-velocity update inherits `zero_vel_stddev_mps` the same way. A caller therefore states what standstill means once, for the whole suite. Tuning them separately was a defect source rather than a feature: tightening `ins`’s gates used to leave the ARS/AHRS – and with them the vertical ZUPT – on untouched defaults, invisibly.

Consequently there are two opt-outs, not one: `auto_zupt_disable` removes every stillness source in the suite, while `auto_zupt_velocity_blind_disable` removes only this velocity-blind fallback and leaves `ins` deciding for everyone. The latter is for a profile the IMU alone cannot tell from a standstill (a noiseless synthetic constant-rate climb or constant-velocity leg in particular: zero sample variance and a gravity-only specific force flatten exactly the trajectory under test). It has the same runtime switch as `ins`’s own detector (`ahrs_set_auto_zaru_disable`), and `nav_suite_set_auto_zupt_zaru_disable` throws both (plus, transitively, `baro_alt`’s vertical zero-velocity update, which has no detector of its own and is driven entirely by these two) with one call.

5.8.1 Tuning the detector for a platform

The shared defaults are sized for a low-cost consumer MEMS IMU (`sensor_defaults.h`). A different sensor class or a different carrier usually needs the thresholds moved, and the two ways to get it wrong leave *different* fingerprints in the replay scoring output:

Arms too rarely. `n auto-zupt` near zero. Biases that the detector exists to observe (notably the ARS/AHRS z-gyro bias, which nothing else can see) drift away unchecked. Typical cause: a platform whose standstill is inherently noisy – a station-keeping multicopter puts more spread on the IMU from its rotors than handheld motion does – so *raise* the variance thresholds.

Arms while moving. The error *scatter* grows while the *bias* stays put. That combination is the giveaway: each spurious update yanks the velocity to zero and the next fix pulls it back, which adds noise without shifting the solution, whereas a genuine accuracy loss normally moves both. Typical cause: a sensor quieter than the defaults assume, so *lower* them.

Both are directly readable from a replay run: the summary line reports `n auto-zupt`, and the resolved thresholds are logged at INFO on startup (`ins: auto-zupt/zaru: window stddev ...`), so there is no need to infer what the filter actually used.

Turn the knobs in this order. The **variance** thresholds (`auto_zupt_static_gyr_stddev_deg`, `auto_zupt_static_acc_stddev_mps2`) first: that is the criterion meant to do the actual rejecting. The **magnitude** bounds (`auto_zupt_static_gyr_deg`, `auto_zupt_static_acc_mps2`) are sized for bias *tolerance*, not for motion rejection, so tightening them to reject motion re-opens the self-locking failure the variance criterion exists to avoid – the two are tuned as a pair. The **velocity** gate (`auto_zupt_max_vel_mps`, `auto_zupt_max_vel_stddev_mps`) only does anything if the receiver actually reports a velocity *with* its covariance; on a position-only stream it never applies, whatever it is set to.

Worked example: `datasets/kfgins`. An automotive fibre-optic-gyro dataset that trips the second failure at the built-in defaults. Its GNSS stream is position-only (no velocity), so the velocity gate never applies and the two IMU criteria decide alone – and a navigation-grade FOG in a smoothly cruising car is far quieter than the consumer MEMS the defaults are sized against. The detector armed mid-drive: 357 triggers instead of 328, attitude-error scatter up thirtyfold, biases unmoved. Sweeping only `auto_zupt_static_gyr_stddev_deg`:

threshold [$^{\circ}\text{s}^{-1}$]	<code>n auto-zupt</code>	roll err. σ [$^{\circ}$]	yaw err. σ [$^{\circ}$]	ARS yaw drift [$^{\circ}\text{min}^{-1}$]
0.25 (default)	357	0.120	0.148	0.775
0.15	335	0.053	0.049	–
0.12	329	0.004	0.026	0.253
0.10	315	0.004	0.026	0.011
0.08	245	0.004	0.026	0.120
0.03	92	0.004	0.026	0.725
≤ 0.02	0	0.004	0.104	–

Two things generalise from the shape of that table. The `ins` metrics have a broad *plateau* (here 0.03 to 0.12), so the value is not picked at the first setting that passes – a threshold chosen on an edge is tuned to one dataset’s noise, not to the platform. And over-tightening is not harmless: in the bottom row, with no triggers at all, the yaw error grows again, because the genuine standstill updates were doing real work.

Where the `ins` plateau leaves the choice open, the *ARS* resolves it. The same key configures every stillness detector in the suite (§5.8), and the *ARS* runs free-yaw here (no magnetometer), so its z -gyro bias is observable only through ZARU – which turns its yaw drift into a smooth optimum in the last column: too high and spurious mid-drive updates corrupt the bias, too low and there are too few updates to determine it. That optimum is visible only on a fine sweep; sampled coarsely the column reads as noise.

5.9 Dead reckoning, re-acquisition, readiness

The filter tracks the time since the last absolute position aiding. Within the coasting window (`max_deadreckoning_sec`, default 10s – roughly how long a consumer-MEMS strapdown position stays usable), incoming fixes fuse normally and the filter keeps dead-reckoning on the IMU. Once the window expires, `ins_is_ready()` degrades to false *and the strapdown stops*: attitude, IMU biases, position and velocity all freeze at their last value instead of dead-reckoning through an outage of unknown length.

The Kalman time update and the measurement fusion stop with it: an expired window makes the whole filter instance *inactive* – nothing is propagated, nothing is fused, and neither the state nor the covariance changes on such an epoch. `ins_is_ready()` has been false since the window expired, so no consumer is entitled to a solution meanwhile; inside `nav_suite` the vertical channel is served by `baro_alt` and attitude by the *ARS/AHRS*, both running the same sensors and neither of them frozen.

The *first usable fix re-acquires* instead of fusing. The re-acquisition:

- Re-anchors position (and velocity, if measured) directly from the measurement – except the vertical position under the barometric height source (§5.5.2), which is re-anchored on the last cached barometer sample instead, through the same datum-referenced conversion the fusion uses. The barometer knows where the platform is. The raw sample carries no filter lag of its own, and its single-sample accuracy becomes the vertical variance. A GNSS fix is moved from the antenna to the IMU first: position by $\mathbf{R}_{b \rightarrow n} \mathbf{l}_b$, velocity by $\mathbf{R}_{b \rightarrow n}(\boldsymbol{\omega} \times \mathbf{l}_b)$ with this epoch’s gyro rate (the frozen filter computed none during the outage), rotated with the attitude the filter will hold after an attitude hint is applied. Without a heading in that hint the yaw is unknown (next item), and only the vertical components are applied.
- *Keeps attitude and IMU biases* – frozen, unchanged since the window expired, and re-leveling would need a static phase a vehicle does not have at a tunnel exit
- *Inflates the variances of everything it keeps* by that state’s process-noise PSD times the frozen duration – the same random-walk growth the time update would have accumulated, computed once from the known elapsed time, capped at the configured initial priors so a re-acquisition after an arbitrarily long outage is at worst as uncertain as a cold start, never worse.
- *Resets the yaw* prior to “heading unknown” ($\sigma \approx 180^\circ$, the same prior the bootstrap uses with no heading source) unless an attitude hint carries an absolute yaw.
- Rebuilds the covariance as a diagonal: fresh position/velocity variances from the measurement noise, inflated attitude/bias variances kept (lost cross-correlations rebuild within a few aided epochs).
- Invalidates the stale history buffer.

The inflation runs *before* the attitude hint and the yaw prior, so an externally supplied variance always wins over this estimate. With `allow_unlimited_deadreckoning` the window and re-acquisition are disabled (for applications that prefer a drifting solution over an unavailable

one). Readiness additionally requires a minimum number of Kalman epochs and runtime after (re-)initialisation.

“Absolute position aiding” is measured on the fusion checks, not on what was actually fused: a position withheld by the decimation of §5.5.7 arrived and was usable, and keeps the window open like any other. Only a position that fails the gates or never arrives opens a gap. This is also why a GNSS *velocity*-only fix – the usual receiver behaviour at a tunnel exit, where Doppler re-converges first – must not refresh the timestamp on its own: the position fix arriving a beat later would find the window un-expired and be run through an ordinary residual fuse it has no chance of surviving, instead of re-anchoring. For the same reason the decimation is suspended while the window is expired.

Aiding *quality* versus aiding *presence*. The coasting window above answers “is aiding arriving at all”. A separate question is whether the aiding that *is* arriving is good enough to carry a 3D solution, and the filter answers it with three independent threshold sets on the same reported GNSS 1σ values:

- `gnss_max_*` – the *fusion* gate: may this individual fix be fused (the loosest set, per measurement);
- `gnss_start_max_*` – the *entry* gate: is the fix stream good enough to enter the 3D solution. The strictest set, and it must hold continuously for `gnss_init_dwell_sec` at ≥ 1 Hz. The velocity threshold carries the decision here (default 0.25 m s^{-1} 1σ per axis): the vertical channel, the accelerometer biases and a moving platform’s yaw observability all lean on the GNSS velocity;
- `gnss_stop_max_*` – the *exit* gate: once every fix over `gnss_stop_dwell_sec` (default 10 s) fails it (default 0.4 m s^{-1} on velocity), the 3D solution is left and the suite falls back to attitude-only. The filter does not merely stop claiming a solution, it stops: it re-arms into the collecting state, so integration and fusion cease and every accessor reports “unavailable” rather than a value drifting off a fix stream already judged too poor. Returning means a full re-bootstrap through the entry gate and dwell, never a single recovered fix. Position, velocity and attitude are re-derived there; the IMU biases carry across with an inflated uncertainty clamped to the cold-start prior, and so does the n-frame origin, so local NED coordinates issued before the outage still mean the same places afterwards.

Only epochs that actually offer aiding are judged, so a plain outage stays the coasting window’s business. The asymmetry – strict and short to enter, loose and long to leave – is the hysteresis of the transition: a premature entry starts the solution from a poor state, a premature exit throws away a converged one that is still the best information available, and a single threshold would let a marginal stream flap the mode. `ins_init` clamps the entry set to at most the fusion set and the exit set to at least the entry set, so a mis-ordered configuration cannot invert it.

5.10 Vertical origin relocation

`ins_shift_origin_down( $\Delta z$ )` moves the n-frame origin along the local down axis without changing the absolute solution. Down is the ellipsoid normal, which is the axis the height is measured along, so the shift touches the origin height alone: h_{origin} decreases by Δz while φ_{origin} and λ_{origin} stay bit-for-bit unchanged, `pos_local[2]` (state and history) shifts by $-\Delta z$, and (φ, λ, h) , covariance and Earth caches are untouched – the shift is deterministic, so no uncertainty is added. The suite uses this to align the ins vertical datum with an already-running barometric filter at bootstrap (§3.3).

6 AHRS: standalone attitude filters

6.1 Model

The AHRS estimates a quaternion $\mathbf{q}_b^n$ and a gyro bias $\mathbf{b}_g$ with an error state of 5 (ARS: $[\delta\phi \ \delta\theta \ \delta\mathbf{b}_g]$) or 6 (AHRS: $[\delta\phi \ \delta\theta \ \delta\psi \ \delta\mathbf{b}_g]$) elements – the same n-frame psi-angle model, sign convention and UDU backend as INS. In ARS (“directional gyro”) mode yaw is integrated from the gyro but never corrected; in AHRS mode it is stabilised by magnetometer heading measurements. The mode is fixed per instance; the runtime- n pattern matches ins’s 15/18 handling. Earth rotation (15° h^{-1}) is *not* compensated here.

Propagation. At IMU rate the quaternion is advanced with the bias-corrected rate $\boldsymbol{\omega} = \tilde{\boldsymbol{\omega}} - \mathbf{b}_g$. The covariance prediction is throttled to 20 Hz with [1, eq. 10.9]

$$\Phi = \mathbf{I} + \mathbf{F}\Delta t, \quad \mathbf{F} = \begin{bmatrix} \mathbf{0} & -\mathbf{R}_{[0:c,:]} \\ \mathbf{0} & \mathbf{0} \end{bmatrix}, \quad \mathbf{G} = \begin{bmatrix} -\mathbf{R}_{[0:c,:]} & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_3 \end{bmatrix}, \quad \mathbf{Q} = \Delta t \begin{bmatrix} \sigma_{\text{gyr}}^2 \mathbf{1}_3 \\ \sigma_{b\text{-RW}}^2 \mathbf{1}_3 \end{bmatrix}, \quad (35)$$

where $\mathbf{R} = \mathbf{R}_b^n$ and $c = 2$ (ARS) or 3 (AHRS) selects the attitude rows.

6.2 Accelerometer leveling

The accelerometer is low-pass filtered (first-order RC, $\alpha = \Delta t / (\tau + \Delta t)$, $\tau = 1 / (2\pi f_c)$, default cut-off 10 Hz) and fused at a throttled rate (default 5 Hz) against the gravity direction:

$$\mathbf{z}_a = \mathbf{f}_{\text{LP}}^b + (\mathbf{R}_b^n)^\top \mathbf{g}^n = (\mathbf{R}_b^n)^\top [\mathbf{g}^n]_\times \boldsymbol{\psi} + \mathbf{v}, \quad (36)$$

so with $\mathbf{g}^n = [0 \ 0 \ g]^\top$ only the roll and pitch columns of $\mathbf{H}$ are non-zero (the yaw column of $[\mathbf{g}^n]_\times$ vanishes) – horizontal accelerations can never feed yaw. The measurement noise is inflated during dynamics:

$$\sigma_{\text{eff}} = \sigma_{\text{acc}} + k \cdot \left| \|\mathbf{f}_{\text{LP}}\| - g \right|, \quad (37)$$

with the *gravity-diff penalty* k (default 8 per m s^{-2}); the default $\sigma_{\text{acc}} = 0.25 \text{ m s}^{-2}$ includes a margin so a moderate disturbance still passes the robust chi-square gate with downweighting, instead of being rejected outright.

Beyond that graded inflation there is a hard gate (`acc_reject_gravity_mps2`): once the low-passed $|\|\mathbf{f}\| - g|$ exceeds it, the leveling update is dropped outright rather than downweighted. A specific force that far from gravity is a manoeuvre or a shock, not a leveling reference, and unlike a persistent reference offset the deviation is inherently transient – so a hard drop cannot deadlock the filter the way it would for GNSS or the barometer (§3.4). A negative value disables the gate.

Attitude-precision restart watchdog. Past a warm-up after init (`restart_warmup_sec`), each tracked attitude axis’ own reported 1σ is checked against `restart_att_stddev_rad`; exceeding it marks the filter uninitialised through the same fail-safe as the health check, e.g. after a long outage of the correcting sensor has let the covariance grow past anything trustworthy. Inside `nav_suite` the instance then re-bootstraps from the live stream on the next epoch, a standalone caller must call `ahrs_init()` again. The warm-up keeps the check from firing while the filter is still converging from a deliberately loose initial covariance. Per-axis thresholds can be disabled individually (negative value), the whole watchdog with `precision_restart_disable`. `baro_alt` carries the same mechanism on its height and vertical velocity (§7.2).

6.3 Magnetometer heading (scalar fusion)

In AHRS mode the body field is de-tilted with the current roll/pitch

$$\begin{bmatrix} h_x \\ h_y \end{bmatrix} = [\mathbf{R}_y(\theta) \mathbf{R}_x(\phi) \tilde{\mathbf{m}}^b]_{1:2}, \quad \psi_{\text{mag}} = \text{atan2}(-h_y, h_x) + \delta_{\text{decl}}, \quad (38)$$

and fused as a scalar heading residual

$$z = \text{wrap}_{\pm\pi}(\psi_{\text{mag}} - \hat{\psi}) = -\delta\psi + v, \quad \mathbf{H} = \begin{bmatrix} 0 & 0 & -1 & 0 & 0 & 0 \end{bmatrix}. \quad (39)$$

Only yaw is observed; magnetic disturbances cannot tilt roll/pitch. Fusion is skipped near gimbal lock, for near-zero fields, and when the de-tilted horizontal component is unusably small relative to the total field. The latter is a measurement-side guard and only bites within about a degree of a dip pole, so it is backed by the position-side exclusion zone (§9.2), which withdraws the heading reference over the surrounding region where the declination is already unusable.

In contrast to ins’s vector fusion (§5.5.10), the scalar heading uses only the field *direction* and is therefore unit-agnostic – a deliberate difference: the standalone AHRS must work with uncalibrated, direction-only magnetometers. Consequently its field-strength gate (same $(\epsilon/\tau)^2$ inflation as ins’s) is *opt-in* and off by default: enabling it requires the magnetometer in μT ; armed unconditionally it would trip on every sample of a direction-only sensor and silently kill heading aiding. ins keeps its gate *opt-out* because μT is contractual for the vector fusion anyway.

6.4 True-north referencing (declination step)

Until a position is supplied the AHRS heading references magnetic north. `ahrs_set_position()` obtains the WMM declination δ_{decl} and applies the *change* in declination as a deterministic left rotation about the n-frame down axis:

$$\mathbf{q} \leftarrow \mathbf{q}_z(\Delta\delta_{\text{decl}}) \otimes \mathbf{q}, \quad (40)$$

so the estimated yaw *steps* to true north instantly instead of slewing there over subsequent magnetometer updates. Covariance and gyro bias are untouched – a known rotation adds no uncertainty. The $+\delta_{\text{decl}}$ term stays in the measurement model, holding the steady state at true north.

The step is only sound while the yaw is *magnetically anchored*, which is what makes the rotation a known one. Inside a dip pole exclusion zone (§9.2) the AHRS suspends magnetometer yaw fusion and keeps its previous declination, so the yaw is carried by the gyro instead. On the position that leaves the zone the new declination is therefore adopted *without* re-framing the attitude, and the fusion pulls the accumulated drift out through the ordinary covariance weighting. Re-framing there would be actively harmful: the declination on the far side of a dip pole differs by tens of degrees (about 80° for a typical crossing), and applying that as a rotation would corrupt an otherwise sound estimate rather than correct it. Roll and pitch are unaffected throughout, only the heading reference is withdrawn.

The one case with no previous declination to keep is a first position that already lies inside a zone, after the yaw has been fused against magnetic north. That yaw is a magnetic heading, and no re-framing will ever make it a true one, so the AHRS widens its yaw variance to “heading unknown” there instead of letting the covariance of that fusion stand. While the attitude-precision restart watchdog is armed the widened value stays just below its yaw threshold, since a restart would discard the zone state together with the position. Otherwise the `nav_suite` attitude hint would hand the magnetic heading to ins as a converged true heading. A yaw that was never fused against the magnetometer keeps its variance.

7 Vertical channel: height systems, `baro_alt` and the offset filter

7.1 Height systems

Three vertical reference systems are in play, and mixing them up is the most common source of confusion in the vertical channel (Figure 4):

Local height h . Height above the NED origin, positive up. Zero at the datum origin *by construction*, whichever sensor started first. This is what a controller consumes (`nav_suite_get_height`), it is available as soon as any vertical source runs, and it is what all parallel filters express themselves in (`ins` as `-pos_local_z`, `baro_alt` as its state h). With a barometer present, `ins`'s own h is itself barometrically fused (§5.5.2), not merely aligned to `baro_alt`'s – the two remain independent estimates of the same physical quantity, sharing the same raw pressure stream and the same datum but not tied together by construction, which is why the suite cross-checks them (REQ-SUITE-019). `nav_suite_get_height` reports `ins`'s h only in FULL mode and `baro_alt`'s otherwise: `ins`'s vertical state stays live through an outage, but `ins` as a whole does not (its horizontal solution is what FULL is about), and `baro_alt` is also the only source before `ins` has bootstrapped at all.

Barometric (ISA) altitude h_{ISA} . The barometer's own height system, Equation (46) (see `baro_alt_get_isa_altitude`). *Not* absolute: it carries the weather-dependent pressure offset and the ISA model error, and it refers to the ISA sea level, not to the ellipsoid. Exposed for diagnostics and for callers that want the raw barometric height.

Ellipsoid height h_{ell} . Absolute, WGS84 (`nav_suite_get_height_ellipsoid`). This is *not always available* – hence the boolean return.

Who defines the datum. The governing rule is: *the datum is fixed once by the first source and never moves. A later source conforms to it at its own initialisation.* So the local height starts at zero in every startup order and – crucially for a height controller – never jumps when a source arrives or drops. The 3D filter's first local solution is therefore always $(0, 0, h)$: horizontal zero (there is no horizontal reference before the first fix), vertical h equal to the current barometric height (0 if no barometer has run yet, non-zero if the vehicle already climbed on the baro before the fix).

Concretely, when the barometer is already running as `ins` bootstraps, `ins` is the one that must join, and *how* depends on who owns the n -frame:

- *Nobody external* (GNSS auto-init, or IMU only): the origin is wherever `ins` happened to bootstrap, i.e. arbitrary. The suite shifts it (`ins_shift_origin_down`) so the local height matches the barometric one; the datum is the mission start and the absolute solution is untouched. Safe because no one else references that frame.
- *An external local-position system* (lighthouse, via `ins_meas_local_pos_t`): that system owns the n -frame, and `ins_shift_origin_down`'s contract requires the external frame to move along, which the suite cannot do. Shifting anyway would put a constant offset on every `local_pos` innovation, and since local references are *skipped* rather than downweighted on outliers, every one would be rejected and the aiding would die silently while the mode still read FULL. The suite therefore reconciles *at the source*: it subtracts a constant offset from every `local_pos` before `ins` sees it, so `ins` bootstraps at $(0, 0, h)$ and stays in lock-step with the external stream. That offset – the external frame's coordinates of the datum origin – is exposed through `nav_suite_get_local_pos_offset()` (§8).

When `ins` is instead already initialised as the first pressure sample arrives, the barometer simply anchors at the current NED height rather than at 0. This is gated on *initialisation*, not on `ins_is_ready()`: a local height may already have moved from the origin during the readiness warm-up, and anchoring at 0 would bake that difference in forever.

The alignment is held per *n*-frame *origin*, not per `ins` instance. A re-bootstrap that anchors a new origin has to re-join the datum; one that keeps the origin carried across a quality-loss re-arm (Fig. 2) must be left alone, because it never left the datum. The difference matters mid-mission: an alignment treats the barometric height as the datum-relative height, which holds when the barometer has just anchored but not after it has drifted against the fix (weather, ISA model error, a tunnel’s own pressure). Shifting a carried origin onto that reading would push the accumulated disagreement into the origin, moving the datum and every local coordinate the consumer already holds – which is what carrying the origin exists to prevent. The carried origin brings the exiting instance’s barometric anchor along with it, so the re-bootstrap takes its initial height from the barometer on that anchor rather than from the bootstrap fix. A drone that loses GNSS, holds its altitude on the barometer and then re-enters the 3D solution therefore does not see a step in its height: the vertical error the fix happens to carry at that moment (several metres right after an indoor or urban outage) never reaches the height channel, which the fix’s vertical row does not aid under the barometric source anyway.

Bridging local to absolute. The bridge is the offset o of §7.3, i.e. *the ellipsoid height of the datum origin*:

$$h_{\text{ell}} = h + o. \quad (41)$$

An absolute height therefore requires GNSS *now* (then `ins` reports it directly), or GNSS for long enough *earlier* to have estimated o – after which the local height source alone carries the absolute height through the outage. With no GNSS ever, o is unknown and `nav_suite_get_height_ellipsoid` returns false while the local height keeps working. This is the whole point of the split: the local height is a *guarantee*, the absolute height is an *offer*.

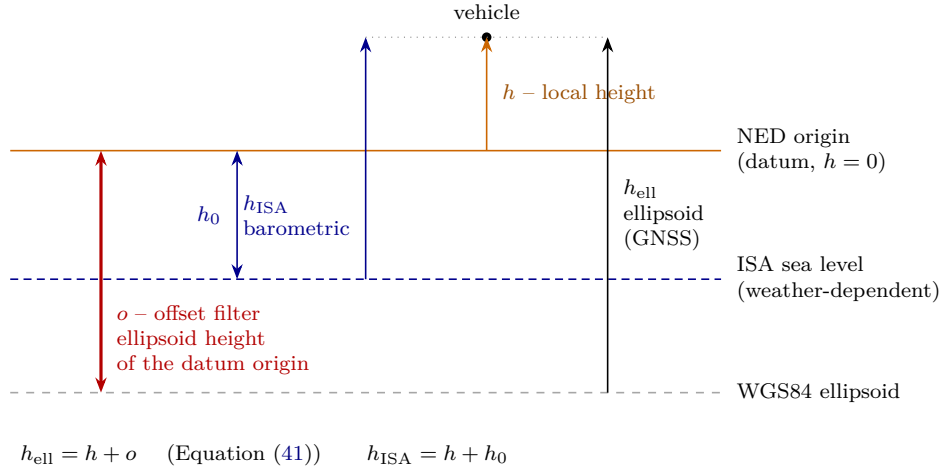


Figure 4: The three height systems. The local height h is measured from the NED origin and is zero at the start point; the offset filter estimates o , the ellipsoid height of that origin, which is what turns a local height into an absolute one during a GNSS outage – and, while GNSS is up and `ins` runs on the barometric height source, what measures how far `ins`’s own vertical datum has drifted away from the ellipsoid, since nothing else re-ties it to GNSS (§7.3). The barometric ISA altitude is a third, non-absolute system, offset from the datum by the anchor constant h_0 . Note that the ISA sea level drifts against the ellipsoid with the weather – which is exactly why o is modelled as a random walk and not as a constant.

7.2 baro_alt: 3-state baro/accelerometer filter

Unlike `ins/ahrs` this is a plain (full-state) *linear* Kalman filter – the model is linear, no error-state formulation is needed. State vector:

$$\mathbf{x} = \begin{bmatrix} h & v & a_b \end{bmatrix}^\top, \quad (42)$$

height above datum h (m, positive up), vertical velocity v , and a slowly varying additive correction a_b to the measured vertical acceleration (absorbing accelerometer bias, attitude error and gravity model error projected onto the vertical axis).

Prediction: accelerometer as control input. Each IMU epoch the body specific force is rotated to NED with the *supplied* attitude quaternion (from `ins` or an AHRS), gravity removed, sign flipped to positive-up:

$$a_{\text{up}} = - \left(\begin{bmatrix} \mathbf{R}_b^n \mathbf{f}^b \end{bmatrix}_z + g \right). \quad (43)$$

The state propagates with constant-acceleration kinematics:

$$\mathbf{x}_{k+1} = \mathbf{\Phi} \mathbf{x}_k + \mathbf{B} a_{\text{up}}, \quad \mathbf{\Phi} = \begin{bmatrix} 1 & \Delta t & \frac{1}{2} \Delta t^2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} \frac{1}{2} \Delta t^2 \\ \Delta t \\ 0 \end{bmatrix}, \quad (44)$$

(note $\mathbf{\Phi}$ couples a_b into h, v , so the effective acceleration is $a_{\text{up}} + a_b$). Process noise in noise-input form, $\mathbf{G} = [\mathbf{e}_1 \ \mathbf{e}_2 \ \mathbf{e}_3]$, $\mathbf{Q} = [\sigma_h^2 \Delta t, \sigma_a^2 \Delta t, \sigma_b^2 \Delta t]$, i.e.

$$\mathbf{Q}_d = \text{diag}(\sigma_h^2 \Delta t, \sigma_a^2 \Delta t, \sigma_b^2 \Delta t). \quad (45)$$

The accelerometer noise enters v ’s own derivative directly (unit weight on $\mathbf{e}_2$), *not* through $\mathbf{B}$: without a noise column of its own, h would inherit v ’s injected uncertainty only one step later, through $\mathbf{\Phi}$ ’s $v \rightarrow h$ coupling. The exact continuous-discrete model for this double integrator additionally places small $Q_{hh} \sim \sigma_a^2 \Delta t^3 / 3$ and $Q_{hv} \sim \sigma_a^2 \Delta t^2 / 2$ terms at every step, which the unit-weight-on- v -only $\mathbf{G}$ leaves at zero; σ_h (`h_process_noise_m_sqrthz`) is a small, independently tunable stand-in for that omitted term – a margin against discretization/model mismatch, not a primary noise source.

σ_h, σ_a (`acc_noise_mps2_sqrthz`) and σ_b (`acc_bias_drift_mps2_sqrthz`) are continuous-time spectral densities ($\sqrt{\text{Hz}}$) scaled by the actual Δt , so the injected variance per second is IMU-rate-invariant; a fixed per-sample sigma would over- or under-inject noise at other rates. The accelerometer is a control input, *not* a measurement: between barometer samples the filter dead-reckons on it.

Barometer measurement. Pressure is converted with the International Standard Atmosphere:

$$h_{\text{ISA}}(p) = 44330 \left(1 - \left(\frac{p}{p_0} \right)^{1/5.255} \right) \text{ m}, \quad p_0 = 101\,325 \text{ Pa}, \quad (46)$$

anchored at the init pressure sample: the anchor’s ISA altitude corresponds to the height h_{init} above the caller’s datum (default 0 = “height above start”), giving the datum zero point $h_0 = h_{\text{ISA}}(p_{\text{anchor}}) - h_{\text{init}}$. Samples are fused as the scalar $z = h_{\text{ISA}}(p) - h_0$ with $\mathbf{H} = [1 \ 0 \ 0]$ through the same robust UDU update – outliers are downweighted (persistent absolute reference). Pressure plausibility is enforced at the boundary (10 kPa to 120 kPa).

Zero-velocity update. A caller-triggered pseudo-measurement (`baro_alt_zero_velocity_update`, e.g. driven by the same stillness detector as `ins`'s own ZUPT/ZARU, §5.8) observes v directly:

$$z = 0, \quad \mathbf{H} = \begin{bmatrix} 0 & 1 & 0 \end{bmatrix}, \quad (47)$$

fused through the same robust UDU update as the barometer (outliers downweighted, not dropped). It is *not* gated on the filter's own v estimate – that estimate is exactly what is being corrected, so gating on it would suppress the update precisely when the accelerometer-only dead reckoning has drifted furthest and needs it most. Beyond bounding v , the update also pulls on a_b through the v/a_b covariance coupling built up by the prediction, so a standstill corrects the acceleration bias even while the barometer is unavailable.

Precision restart watchdog. Past a warm-up period after init (`restart_warmup_sec`), the filter's own reported h and v 1-sigma (from the UDU factors) are checked against `restart_h_stddev_m / restart_v_stddev_mps`, if either is exceeded the filter is marked uninitialized (the same fail-safe mechanism as the general health check), e.g. after a long barometer outage has left the accelerometer-only dead reckoning too uncertain to trust. `nav_suite` then re-bootstraps the vertical channel from the live stream, a standalone caller must call `baro_alt_init` again. Either threshold can be disabled individually (negative value), or the whole watchdog turned off (`precision_restart_disable`).

7.3 `local_gnss_alt`: 1-state offset filter

A scalar random-walk Kalman filter estimates

$$o = h_{\text{GNSS,ellipsoid}} - h_{\text{local}}, \quad (48)$$

the ellipsoid height of the datum origin (§7.1). Both heights refer to the IMU point: the fix reports the antenna, so the suite first moves it by the down component of the GNSS lever arm, rotated with its best available attitude, $h_{\text{GNSS,ellipsoid}} = h_{\text{antenna}} + (\mathbf{R}_{b \rightarrow n} \mathbf{l}_b)_D$. Without that, o would absorb the lever arm and the absolute height would jump by it whenever `nav_suite_get_height_ellipsoid` switches between `ins` and $h_{\text{local}} + o$. The filter is *agnostic* to where the local height comes from: with the barometric source o lumps the weather-dependent pressure offset, the geoid undulation and the ISA model error, with an external local height (e.g. a lighthouse system) it lumps the geoid undulation and the datum's own height. Propagation $\sigma_o^2 \leftarrow \sigma_o^2 + \sigma_{\text{RW}}^2 \Delta t$ (default $0.3\text{m}/\sqrt{\text{s}}$: slow enough to average pair noise, fast enough for the ISA-model error, confidence decays during outages). It is that term and not the weather that sizes the default. Weather drift of metres per hour would be satisfied by an order of magnitude less; the ISA error is instead a fraction of the height *excursion* – a tenth of it is ordinary, the model assuming a sea-level temperature the real atmosphere rarely has – so a few hundred metres of terrain climbed in a few minutes move o by tens of metres in those minutes. A random walk in *time* can only bound a drift whose driver is altitude, never model it, and the bound has to stay wide enough that the chi-square screening below still reads such a drift as drift: too slow a walk turns real, persistent drift into a stream of apparent outliers, and the downweighting then holds o back precisely when it has the most catching up to do. Update with pair variance $R = \sigma_{\text{local}}^2 + \sigma_{\text{GNSS}}^2$ and the same chi-square downweighting, written out directly ($n=1$, no UDU needed):

$$K = \frac{\sigma_o^2}{\sigma_o^2 + R}, \quad o \leftarrow o + K \delta z, \quad \sigma_o^2 \leftarrow (1 - K) \sigma_o^2. \quad (49)$$

Both stddevs are additionally derated by a configurable inflation factor and the fusion is decimated to a configurable minimum interval, which together keep a single pair from nudging the offset around.

Which local height to feed it with. The suite feeds the filter from the source that *survives a GNSS outage* – the barometer if it is running, otherwise the `ins` local height – and *not* from the momentarily most accurate one. While GNSS is up the `ins` local height is by far the better estimate, but pairing it against GNSS would yield a drift-free o that is useless the moment it is needed, because the height it would then be added to (the barometer’s) carries a weather drift o never learned to absorb. The same reference evaluates Equation (41), so offset and height always refer to one source.

Correcting the datum drift. Under the barometric height source `ins`’s absolute height is only as good as its vertical *datum*, and that datum is anchored once at bootstrap and never re-tied to GNSS (REQ-NAV-055): the barometer’s whole absolute error accumulates in it for the rest of the run. Its height *relative* to that datum stays excellent, so the absolute accessor keeps `ins` and subtracts only how far o has moved since the datum was fixed. Two details make that safe. The anchor *tracks* o while the filter is still converging and freezes when it settles – a snapshot taken mid-convergence books the remaining convergence as drift that never happened, and being a constant, that error becomes a permanent bias. And only the part of the drift exceeding one σ of its own estimate is applied, less that margin: `ins` is the better estimate whenever the datum has *not* moved, which is most missions most of the time, so replacing its decimetre-grade height with $h + o$ outright would buy back tens of metres on a long drive at the price of the offset filter’s metre-grade noise on every epoch everywhere else. When the fix is delayed, the local height is extrapolated to the GNSS time of validity with the estimated climb rate.

Datum coupling. The offset is tied to the datum and would have to move with any datum relocation. Under the “the datum never moves” rule (§7.1) that never arises: when the suite relocates the *ins* origin onto the (stationary) baro datum at a GNSS bootstrap, the datum the offset refers to has *not* moved, so the offset is left untouched. Shifting it there would put a permanent Δz error on every absolute height.

8 Frame offsets and coordinate conversion

A vehicle rarely lives purely in the local n -frame: an indoor drone must relate the Lighthouse system’s frame to the suite’s datum, and an outdoor drone must turn WGS84 waypoints into the local frame its controller flies in. The suite exposes both bridges as accessor pairs, one per side.

Local-position frame (§7.1). An external local-positioning system reports in its own frame, whose origin is arbitrary. The suite reconciles it by subtracting a constant offset $\mathbf{o}_{lp}$ from every `local_pos` before `ins` consumes it, so `ins` always solves at $(0, 0, h)$ relative to the datum. $\mathbf{o}_{lp}$ is the external frame’s coordinates of the datum origin, exposed by `nav_suite_get_local_pos_offset()`:

$$\mathbf{p}_{\text{external}} = \mathbf{p}_{\text{local}} + \mathbf{o}_{lp}, \quad \mathbf{p}_{\text{local}} = \mathbf{p}_{\text{external}} - \mathbf{o}_{lp}. \quad (50)$$

Its vertical component aligns to the barometric height that was already established (which is what removes the height jump when the local system comes online late), the horizontal component is the system’s report at first sample. The offset is tracked until `ins` locks the datum at bootstrap, then frozen, and survives `ins` resets.

WGS84 (the GNSS side). `nav_suite_local_to_wgs84()` and its inverse `nav_suite_wgs84_to_local()` convert between a local NED position (relative to the datum origin) and

geodetic lat/lon/ellipsoid-height. Both directions use the same curvature-radii mapping $\mathcal{M}$ that the filter moves its own anchor with (§5.3), applied around the geodetic origin $(\varphi, \lambda, h)_{\text{origin}}$:

$$(\varphi, \lambda, h) = (\varphi, \lambda, h)_{\text{origin}} + \mathcal{M}^{-1}(\mathbf{p}_{\text{local}}^n), \quad \mathbf{p}_{\text{local}}^n = \mathcal{M}((\varphi, \lambda, h) - (\varphi, \lambda, h)_{\text{origin}}). \quad (51)$$

There is no detour through ECEF here: routing it that way would answer the same question with a different approximation than the filter’s own position bookkeeping uses, so the local frame and the absolute solution would disagree at the metre level far from the origin. This is the GNSS-side counterpart of the local-position offset: it turns the local solution into WGS84 for telemetry, or WGS84 waypoints into the frame a controller flies in.

Both WGS84 converters, like `nav_suite_get_height_ellipsoid` (§7.1), are available only once a real GNSS fix has anchored the origin to WGS84 – established by any usable fix, at bootstrap or by fusion. Without it the origin is only the prescribed init position (a rough indoor guess, or a lighthouse frame), so the converters return false rather than emit a fictitious absolute coordinate.

9 World Magnetic Model lookup

`magnetic_model` provides declination D , inclination I and total field strength F for a position and decimal year, from generated lookup tables (`wmm_lut.h`, produced offline by pygeomag-based generators) instead of an online spherical-harmonics evaluation: fixed number of table lookups per call, no heap, bounded time. Declination is stored on a fine grid and interpolated bilinearly in space and linearly in time between the two model epochs, inclination and field strength use a coarser grid, bilinear in space only (they change slowly over a 5-year epoch). The NED reference vector is assembled self-consistently as

$$\mathbf{m}^n = F \begin{bmatrix} \cos I \cos D \\ \cos I \sin D \\ \sin I \end{bmatrix}. \quad (52)$$

Consumers: `ins_set_magnetic_model_from_position()` builds the full reference vector (so `ins`’s vector fusion yaw is true-north) and arms the field-strength gate, `ahrs_set_position()` uses D for the deterministic declination step and F for the opt-in gate. Like `geodetic_toolbox`, the module carries no filter policy – “is this measurement trustworthy” is decided in the filters.

9.1 Declination is a wrapped angle

D lives in $(-180^\circ, +180^\circ]$, so two neighbouring grid nodes can sit on opposite sides of that cut while being a few degrees apart (for example -176.5° and $+178.5^\circ$). Interpolating the tabulated numbers directly would then run the long way round and return a value off by up to 180° . Both the spatial and the temporal interpolation therefore put the operands on a common branch first: the secular change per node is taken the short way, and the three neighbouring nodes are shifted next to the first before the bilinear blend. The affected cells lie along the agonic line trailing each dip pole, which is where the filters are most in need of a trustworthy heading reference.

9.2 Dip pole exclusion zones

At a magnetic dip pole the horizontal field vanishes, so D is not merely inaccurate there but ill-conditioned: it rotates through the full circle around the pole, and no regular grid resolves that. The exact model puts the residual interpolation error at roughly 179° within a degree of a pole; outside the 7° exclusion radius the worst case is about 10° , for 0.75 % of the Earth’s

surface. A magnetometer is no better off, because the horizontal component it needs has fallen to a few percent of the total field.

`magnetic_dip_pole_distance_deg()` reports the great-circle distance to the nearest dip pole and `magnetic_heading_reference_valid()` is false within `MAGNETIC_DIP_POLE_EXCLUSION_DEG` of one. Inclination and total field strength stay usable inside a zone, only the heading reference is withdrawn. Three properties are worth stating explicitly:

Great circle, not latitude and longitude. Near the northern pole a degree of longitude covers about a twelfth of the ground a degree of latitude does (46 km against 555 km at 85° N), so a box in coordinates would be wrong by an order of magnitude in one direction.

Not a latitude limit. The southern dip pole sits near 64° S, far from the geographic pole, so no cutoff on latitude separates the zones from ordinary terrain without discarding the whole subantarctic.

The count is data, not an invariant. The number of dip poles and their positions are measured per epoch by the generator and tabulated (`WMM_DIP_POLE_COUNT`), and the runtime walks whatever it finds. Today’s field has two, both drifting, the northern one by roughly 33 km per year. That is a property of the present dipole dominance rather than something the code may assume.

No interpolation in time. Positions are tabulated once at mid-epoch and the query takes no year, unlike the declination. Over five years the poles move about 1.5°, so a fixed mid-epoch position is at most $\sim 0.8^\circ$ off at either end – smaller than the slack in the radius, and smaller than the uncertainty in picking that radius in the first place. Interpolating would compute a correction finer than the quantity it feeds. The generator measures the drift on every regeneration and refuses to emit positions that exceed the budgeted margin.

Because the poles move, the exclusion radius wants re-examining whenever the tables are regenerated. `magneticmodel/wmm_error_analysis.py` re-measures the shipped table against the exact model and reports the residual error outside the zones for exactly that purpose. Sample it at 0.5° or finer: the worst points sit on a thin ridge at the exclusion boundary, and a coarser grid steps over them and understates the maximum.

10 Design decisions

This section collects the load-bearing decisions and their rationale in one place, the detailed context is in the sections above.

UDU factorisation instead of a full covariance. Single-precision hot path requires it, $P \succeq 0$ by construction, roughly double effective precision (§4).

Error-state formulation (ins, ahrs). The nominal state absorbs the nonlinear mechanisation, the Kalman filter sees small, nearly linear errors. Attitude errors stay far from singularities, the quaternion is corrected multiplicatively.

Downweight vs. skip per measurement class. Persistent absolute references are down-weighted (no deadlock on persistent offsets), high-rate local references are skipped (glitches are cheap to drop) – §3.4.

Running mean instead of median for the ZARU average. A median needs a buffer and selection ($O(k)$ memory, data-dependent work in the per-epoch path), the running mean is $O(1)$. WCET outranks statistical elegance here – do not “improve” this.

Throttled covariance prediction. Error dynamics are slow, predicting at IMU rate buys nothing but cycles (§5.4). Same reasoning throttles the AHRS covariance (20 Hz) and its acc/mag fusion rates.

Vector vs. scalar magnetometer fusion. `ins` fuses the field vector (with yaw-only Jacobian columns) – `µT` is contractual, enabling the magnitude gate by default and hard-iron bias estimation. The AHRS fuses a scalar heading – unit-agnostic by design for direction-only sensors, so its magnitude gate is opt-in (§6.3).

Hard-iron bias inside the filter, not outside. Separability from yaw requires the yaw/bias cross-covariance, which only the filter maintains (§5.5.10). Runtime-*n* (15/18) instead of a second code path or a compile-time fork.

Re-acquisition keeps attitude and biases, but not the yaw *prior*. After a long outage position/velocity are gone but attitude/biases are the best-preserved states. Resetting them would require a static re-leveling phase the vehicle does not have at a tunnel exit. Yaw is the exception: with the state frozen, the platform’s own rotation over the outage never entered it, so without an absolute heading its *variance* goes back to “unknown” – otherwise the heading correction ends up in the gyro *z* bias (§5.9).

History-anchored delayed fusion. Residual at time of validity, covariance update on the current factors – valid because the error state barely moves over a few hundred milliseconds (§5.6).

Float states, double anchor. Local float *n*-frame keeps the hot path fast on FPUs without double units, the double absolute anchor with incremental bookkeeping avoids the metre-scale quantisation of absolute float coordinates (§5.3).

Independent accelerometer noise defaults per filter. The three “accelerometer” quantities are physically different measurements (raw VRW vs. leveling correction vs. attitude-projected vertical acceleration), sharing one constant regressed the vertical filter empirically (§3.6). Gyro defaults *are* shared – same physical sensor in the suite.

Deterministic declination step. Referencing to true north is a known rotation, so it is applied as an instantaneous re-frame of the nominal quaternion, not learned through the fusion (which would slew the heading over seconds and, transiently, disagree with the covariance). It is skipped on leaving a dip pole exclusion zone, where the yaw was carried by the gyro rather than the field and the rotation would therefore not be a known one (§9.2).

Dip pole zones are excluded by distance, not by latitude. The southern dip pole sits near 64° S, so a latitude cutoff cannot separate the degenerate region from ordinary terrain. The number of poles and their drifting positions are measured per epoch and tabulated rather than assumed in code, because both are properties of the present field and not invariants.

Conservative defaults everywhere. A zero/unset noise field never means “no noise” – it means “use a conservative consumer-MEMS placeholder”. An overconfident filter fails silently, an overly cautious one is just suboptimal.

Barometric height fused directly in `ins`, not switched in from `baro_alt` on GNSS loss.

Two designs were considered: (a) keep `ins`’s height GNSS-only and let `nav_suite` switch the reported height to `baro_alt` whenever GNSS aiding drops, or (b) have `ins` itself fuse barometric height (§5.5.2), replacing GNSS-derived height permanently once selected at bootstrap.

(a) was rejected on two counts. A discrete switch between two independently-drifting estimates is a discontinuity by construction: no source stays a fixed distance from another over an arbitrary mission, so the step size is unbounded in general, not just in a corner case (on one real dataset the two disagreed by over ten metres after half an hour, entirely within continuous GNSS coverage). A hysteresis band only changes how rarely the jump happens. More fundamentally, splicing `ins`’s horizontal state onto `baro_alt`’s independently-estimated vertical state discards the cross-covariance a single error-state filter maintains (an attitude error couples into all three position axes at once, §5.2), so the spliced vector is no longer a coherent minimum-variance estimate. (b) has neither problem: the height source is fixed once at bootstrap, so `ins`’s vertical state never switches mid-mission, and it stays inside `ins`’s own state and covariance.

(b) does not remove the accessor-level handover: `nav_suite_get_height` still falls back to `baro_alt` outside FULL mode, because `ins` as a whole can be unavailable while a height is still wanted. What (b) changes is its *character* – both sides are then the same barometric quantity on one datum, so the step is the disagreement between two estimators of it rather than the gap between two height systems. That disagreement is cross-checked every epoch and warned about (REQ-SUITE-019).

Two prices. `ins`’s height then depends on the barometer’s absolute accuracy (weather/ISA model error) rather than GNSS’s. That is the right trade for the height *relative* to `ins`’s own vertical datum, where the barometer is the better instrument and GNSS’s vertical accuracy is routinely the worse of the two. It is the wrong trade for the *absolute* height, and not by a little: the datum is anchored once at bootstrap and never re-tied to GNSS, so the barometer’s whole absolute error accumulates in it without bound. On a 68 min drive over 170 m of terrain that reached 19 m RMS against a receiver whose own vertical spread at standstill measured 0.08 m – the premise that GNSS is the worse vertical source does not hold for a good receiver. The absolute accessor therefore *corrects* `ins`’s height under this source by the offset filter’s estimate of how far the datum has since drifted (§7.3), which is exactly the quantity that filter exists to track. It corrects rather than replaces: `ins` remains the better estimate whenever the datum has not moved. The relative accessor is unaffected. The second price, accepted: because the selection is never re-evaluated, a barometer failing mid-mission leaves the vertical channel without an absolute reference while GNSS keeps aiding the horizontal solution: re-evaluating per epoch would reintroduce exactly the discontinuity (a) was rejected for, so the case is mitigated rather than fixed – the sample must be current at bootstrap, not merely present, and the ensuing aiding gap is reported explicitly (§5.7).

11 Dataset configuration: `config.yaml`

A dataset directory carries the neutral CSV streams (`imu.csv`, `ref.csv`, and optionally `gnss.csv`, `mag.csv`, `baro.csv`, `speed.csv`, `heading.csv`, `ranges.csv`, the column contract lives in `datasets/replay_format.py`) plus one `config.yaml` that tells the harness *how* to run the filter: aiding source, initialisation mode, the sensor noise model, sensor calibration and lever arms, aiding-quality gates, the (non-filter) scoring thresholds, and optional per-stream CSV filename overrides. Every key is optional except the IMU noise model (and `gnss: pos_stddev_fallback_m/vel_stddev_fallback_mps` under `aiding: ref`, see below), a value of 0 (or an all-zero list) means “use the built-in default / no-op”, never “force zero”. An unknown key is a hard error in both harnesses, naming the offending key: a silently ignored key is indistinguishable from one that had no effect, so a typo would otherwise produce a full run, a score and a plot computed with a configuration nobody asked for.

“Known” here means the *schema*, not what a given harness happens to consume. A dataset

directory is read by more than the two replay harnesses – `check_simulated.py` owns `score:` `lim_groves_pos_rms_factor`, `crazyflie_reader.py` owns the `origin:` and `crazyflie:` sections – so each harness also accepts, and ignores, the parts it does not consume, from a list the two keep identical. Otherwise a dataset would have to choose which of its tools it validates against, and deleting the offending key would silently disarm the tool that does read it. A typo inside a section the harness *does* own remains an error, and a section earns the exemption only once some tool actually reads it.

The live receiver `tools/insrcv.c` reads the same file and is one step more lenient: it implements a *subset* of the schema, and a key it does not consume is named on the console and then ignored instead of aborting the run. Refusing to start a receiver over a `score:` section would be useless, but a tuning value that silently has no effect misleads there as much as in a replay – so keys describing the replay harness itself (`score:`, `inputs:` and the `name/aiding/init` labels) pass unremarked, and everything else the receiver does not implement is reported, with the count repeated in its config summary line.

A representative annotated example (keys not shown fall back to their defaults):

```
name: my-dataset           # label for plots / the summary
aiding: gnss               # gnss / ref / none
init: auto                 # auto / ref
automotive_mode: 0         # 1: derive yaw from GNSS course-over-ground
chi2_disable: 0            # 1: disable outlier downweighting (diagnostics)
chi2_reject_alpha: 0.0     # 0 -> outlier gate, e.g. 0.05 -> 95% global
allow_unlimited_deadreckoning: 0 # 1: never expire the IMU-only coasting window
max_deadreckoning_sec: 0.0 # IMU-only coasting budget [s], 0 -> default 10 s
baro_height_disable: 0     # 1: never select the barometric height source,
                           # keep GNSS position's vertical row. For a
                           # gnss_pos stream that is not satellite GNSS
                           # and is more accurate vertically than a
                           # barometer (e.g. a Lighthouse/UWB rig), and
                           # for a barometer whose static port does not
                           # see ambient pressure: a sensor inside a car
                           # cabin moves tens of metres with ventilation
                           # and speed, far past any correction GNSS
                           # could still apply, since the vertical row
                           # is dropped for the whole run.

gyro_bias_window_sec: 3.0 # static window to seed the initial gyro bias
                           # (harness-only heuristic, no library default;
                           # 0 -> no seed at all)

auto_init_window_sec: 0.0 # IMU window ins levels over during auto-init [s]
                           # 0 -> default, raise it for a low IMU rate

imu:                       # sensor noise model (REQUIRED) + calibration
  gyr_psd: 3.0e-11         # gyro white noise PSD [(rad/s)^2/Hz]
  acc_psd: 4.0e-06         # accel white noise PSD [(m/s^2)^2/Hz]
  gyr_bias_rw: 1.4e-06     # gyro bias random walk [rad/s/sqrt(s)]
  acc_bias_rw: 3.0e-04     # accel bias random walk [m/s^2/sqrt(s)]
  # optional fixed calibration, corrected = M*(raw - bias):
  gyr_fixed_bias: [0,0,0]  # removed permanently, never estimated [rad/s]
  acc_fixed_bias: [0,0,0]  # [m/s^2]
  gyr_misalignment: [1,0,0, 0,1,0, 0,0,1] # col-major 3x3 (0s->identity)
  acc_misalignment: [1,0,0, 0,1,0, 0,0,1]
  mount_rpy_deg: [0, 0, 0] # board attitude in the vehicle frame [deg]
  # extra process-noise margin on top of the acc/gyro-driven pos/vel/rpy
  # noise above, 0 -> default. The rpy default assumes a NON-RIGID mount,
  # where attitude model error dwarfs the gyro's random walk; a rigid mount
  # wants far less and sets it here.
  pos_pred_stddev_m_sqrts: 0.0 # [m/sqrt(s)]
  vel_pred_stddev_mps_sqrts: 0.0 # [m/s/sqrt(s)]
  rpy_pred_stddev_rad_sqrts: 0.0 # [rad/sqrt(s)]
```

```

# Stillness: ONE set for the whole suite. It reaches ins, both AHRS
# instances and (through them) baro_alt's vertical ZUPT -- the filters
# implement it separately, they are not tuned separately.
# Every field 0 -> the library's own default.
#
# pseudo-measurement stddev: how hard a detected standstill is trusted
zero_vel_stddev_mps: 0.0      # [m/s] (ins + baro_alt vertical ZUPT)
zero_rot_stddev_deg: 0.0      # [deg/s] (ins + ARS/AHRS ZARU)
# PRIMARY stillness criterion: per-axis RMS sample stddev of the raw
# IMU over a short window (bias-invariant, hence primary)
auto_zupt_static_gyr_stddev_deg: 0.0 # [deg/s]
auto_zupt_static_acc_stddev_mps2: 0.0 # [m/s^2]
# loose magnitude bounds beside it
auto_zupt_static_gyr_deg: 0.0 # max |gyro| considered stationary [deg/s]
auto_zupt_static_acc_mps2: 0.0 # max ||f| - g| [m/s^2]
# velocity gate: applied only while a recent GNSS velocity exists that
# is itself precise enough (no such fix -> the IMU decides alone; the
# filter's own velocity state is never used, that would be circular)
auto_zupt_max_vel_mps: 0.0 # max |GNSS velocity| to arm [m/s]
auto_zupt_max_vel_stddev_mps: 0.0 # max GNSS vel 1-sigma to trust it [m/s]
# timing
auto_zupt_dwell_sec: 0.0 # stillness required before a trigger [s]
auto_zupt_min_interval_sec: 0.0 # min time between triggers [s]
# opt-outs
auto_zupt_disable: 0 # 1 -> no stillness detection anywhere in the suite
auto_zupt_velocity_blind_disable: 0 # 1 -> only the ARS/AHRS's own
# detector off (it cannot tell constant-velocity
# cruise from a standstill); ins keeps deciding

gnss:
  enable: 1 # 0 -> the fixes are still read and counted,
            # but none of them aids the filter
  leverarm_frd: [0,0,0] # IMU->antenna lever arm, body FRD [m]
  delay_ms: 0 # assumed fixed fix latency [ms]
  max_horizontal_pos_stddev_m: 0.0 # fusion gate: ignore fixes worse than
  max_vertical_pos_stddev_m: 0.0 # this (per fix, see below), 0 -> ins.c's
  max_horizontal_vel_stddev_mps: 0.0 # own default, which is the accuracy CAP
  max_vertical_vel_stddev_mps: 0.0 # below (120 m / 60 m/s): by default no
                                # fix is rejected for its reported
                                # accuracy alone, it is capped and fused.
                                # Set these below the caps for a hard gate.
  # 3D solution ENTRY gate (strict) + its dwell, 0 -> default:
  start_max_horizontal_pos_stddev_m: 0.0 # [m]
  start_max_vertical_pos_stddev_m: 0.0 # [m]
  start_max_horizontal_vel_stddev_mps: 0.0 # [m/s]
  start_max_vertical_vel_stddev_mps: 0.0 # [m/s]
  init_dwell_sec: 0.0 # how long the stream must stay entry-quality
  init_dwell_disable: 0 # 1: enter on the first good fix
  # 3D solution EXIT gate (loose) + its dwell, 0 -> ins.c's own default:
  stop_max_horizontal_pos_stddev_m: 0.0 # [m]
  stop_max_vertical_pos_stddev_m: 0.0 # [m]
  stop_max_horizontal_vel_stddev_mps: 0.0 # [m/s]
  stop_max_vertical_vel_stddev_mps: 0.0 # [m/s]
  stop_dwell_sec: 0.0 # bad-fix run before the 3D solution is left
  stop_disable: 0 # 1: never leave 3D on GNSS quality alone
  min_delay_ms: 0 # fusion rate limit: minimum time between two
                  # fixes the filter actually spends. A fix
                  # arriving sooner is skipped whole (both blocks)
                  # and counted in n_gnss_rate_limited; suspended
                  # while the coasting window is expired, so a
                  # tunnel exit is never paced. A GNSS error
                  # decorrelates over seconds, not over the fix
                  # interval, so fusing a 50 Hz stream at 50 Hz

```

```

# presents the same information ten times and
# collapses the covariance.
# 0 -> default (100 ms, 10 Hz), <0 -> no limit
pos_decimation: 0
# fuse the position on every Nth epoch that
# offers BOTH a usable position and velocity,
# the velocity alone on the other N-1;
# 1 -> off, 0 -> ins.c's own default, which is
# OFF: min_delay_ms above covers the same
# over-fusing without withholding a block, and
# running both thins the position twice

pos_stddev_fallback_m: [0,0]
# [hor,ver] used if reported cov is 0;
# REQUIRED (no default) if aiding: ref,
# where it is the ONLY noise the
# truth-synthesized fix gets

vel_stddev_fallback_mps: 0.0
# same, required if aiding: ref
# The conditioning knobs below weight the FUSION only. The
# fix-quality gates (max_*, start_max_*, stop_max_*, and the auto-ZUPT
# accuracy gate) keep grading the stddev the receiver reported, so
# downweighting a noisy receiver never silently stops the filter from
# using it. Scale them freely against the gates. The pipeline runs
# envelope -> scale -> height scale -> floor -> cap -> manoeuvre term
# and is described in REQ-NAV-038/041/071/072/073.
acc_envelope_tau_sec: 0.0
# asymmetric tracking of the REPORTED accuracy:
# a rise takes effect at once, a fall only decays
# out with this time constant. A receiver's
# accuracy figure returns to nominal well before
# a re-acquired fix has actually settled.
# 0 -> default (5 s), <0 -> off

pos_cov_scale: 0.0
# multiply reported pos stddev (0->1)
pos_cov_scale_height: 0.0
# extra downweight of the pos HEIGHT axis (0->1)
vel_cov_scale: 0.0

pos_stddev_floor_hor_m: 0.0
pos_stddev_floor_ver_m: 0.0
# floor per-axis stddev (independently,
# NOT correlation-preserving like
# pos_cov_scale above: raises N/E/D
# diagonal entries alone, so a
# floored axis picks up the effect of
# extra independent noise).

vel_stddev_floor_hor_mps: 0.0
vel_stddev_floor_ver_mps: 0.0
# Unlike the scales, 0 means "use ins.c's
# default", NOT "no floor". The defaults
# (0.5/1.0 m, 0.3/0.5 m/s) describe a
# single-frequency (L1) standalone
# receiver, which is what an
# unconfigured filter should assume: a
# receiver claiming better than such a
# solution can physically deliver is the
# normal case, not the exception.
#
# A BETTER source must say so, or the
# floors throw its precision away. Set
# each floor to what that source really
# delivers, e.g. for RTK fix:
#   pos_stddev_floor_hor_m: 0.02
#   pos_stddev_floor_ver_m: 0.03
#   vel_stddev_floor_hor_mps: 0.03
#   vel_stddev_floor_ver_mps: 0.05
# A NEGATIVE value switches a floor off
# entirely, i.e. trusts the reported
# covariance unconditionally. Prefer
# real numbers over -1: an RTK receiver
# dropping to float or single still
# reports optimistically, and a floor
# sized for the fix mode is what keeps
# that drop from being believed.

```

```

pos_stddev_cap_hor_m: 0.0
pos_stddev_cap_ver_m: 0.0
vel_stddev_cap_hor_mps: 0.0
vel_stddev_cap_ver_mps: 0.0

vel_noise_acc_scale_hor: 0.0
vel_noise_acc_scale_ver: 0.0

vel_noise_acc_window_sec: 0.0

```

```

# upper clamp, same 0 -> default /
# <0 -> off convention (120 m, 60 m/s).
# Bounds what an unboundedly bad reported
# accuracy can do to the 32-bit hot path,
# and is what lets max_* default to
# "fuse everything". A cap below the
# floor of the same axis group is raised
# to that floor at init.
# manoeuvre-dependent EXTRA velocity noise
# in [m/s] per [m/s^2] of n-frame
# ANTENNA acceleration (horizontal norm
# on N/E, |Down| on D), added after the
# cap. A Doppler velocity averages over
# the receiver's measurement interval
# while the state is instantaneous; under
# acceleration the two differ by about
#  $a \cdot T/2$ , which the reported accuracy
# cannot describe. The acceleration is
# the antenna's, i.e. the body's plus the
# centripetal term  $\omega \times (\omega \times l)$  of
# the leverarm_b below (REQ-NAV-076), so
# a platform turning about its own IMU
# still pays for the arc its antenna
# swings through. With no lever arm that
# addition is exactly zero.
# 0 -> default, <0 -> off. The defaults
# (0.20 / 0.30) ARE that half-interval
# for  $T = 400$  ms, four times the rate
# min_delay_ms paces fusion to (margin:  $T$ 
# belongs to the receiver's velocity
# algorithm, not its output rate), plus a
# margin on the vertical axis for its
# one-sided satellite geometry. Fixed
# constants: NOT derived from the
# observed fix rate at runtime. At a
# 45 deg lean that adds roughly 2 m/s,
# more than six times the horizontal
# velocity floor; at rest
# it adds nothing, so the standstill
# regime is untouched. Raise them for a
# receiver whose velocity is
# time-differenced carrier phase over a
# long output interval ( $T$  is then the
# full interval). Disable them for a
# source with no measurement interval at
# all, e.g. a pose tracker or a simulator.
# window the acceleration above is averaged
# over: the SAME  $T$  the scales are half of.
# The error priced is what averaging over
#  $T$  does to a velocity, so the
# acceleration over  $T$  is what sets it.
# Without it a manoeuvre that just ended
# goes unpriced on the fix that still
# carries its error, and airframe
# vibration is priced as if it were a
# manoeuvre (measured: a zero-mean
#  $4 \text{ m/s}^2$  oscillation adds  $4 \text{ (m/s)}^2$ 
# instantaneous vs 0.01 windowed). The
# VECTOR is averaged, so a zero-mean
# swing cancels exactly. The rotation
# half of the antenna acceleration uses
# the same window but averages
#  $\omega \times \omega'$ , which is quadratic and

```

```

# therefore does NOT cancel for a
# platform that turns back and forth.
# 0 -> default (200 ms), <0 -> use the
# instantaneous sample

init_stddev:          # initial-state 1-sigma (each 0 -> built-in default)
  pos_init_stddev_m: 0.0          # [m]
  vel_init_stddev_mps: 0.0        # [m/s]
  rpy_init_stddev_rad_deg: 0.0    # [deg]
  yaw_init_stddev_rad_deg: 0.0    # [deg] (0 -> rpy_init_stddev_rad_deg)
  acc_bias_init_stddev_mps2: 0.0  # [m/s^2]
  gyr_bias_init_stddev_rps_deg: 0.0 # [deg/s]

init_hint:            # known initial roll/pitch/yaw for auto-init
                      # (init: auto only), roll/pitch used together,
                      # yaw independent, each 0 stddev -> no hint

  roll_deg: 0.0
  pitch_deg: 0.0
  rpy_stddev_deg: 0.0        # 0 -> no roll/pitch hint (usual: gravity levels it)
  yaw_deg: 0.0               # e.g. a known launch heading
  yaw_stddev_deg: 0.0        # 0 -> no yaw hint (usual: stays unknown until aiding)

mag:
  enable: 0                # replay default (it opens mag.csv);
                          # tools/insrcv.c defaults to 1
  stddev_ut: 0.0           # heading-measurement noise [uT] (0 -> default)
  min_delay_ms: 0          # min. time between fusions (0 -> default,
                          # negative -> fuse every sample)
  wmm_year: 2025.0         # decimal year for the declination lookup
  estimate_bias: 0         # 18-state hard-iron bias
  bias_init_ut: 0.0        # INITIAL hard-iron stddev [uT], 0 -> default
  bias_rw_ut_sqrts: 0.0    # hard-iron random walk [uT/sqrt(s)], 0 -> default
  fixed_bias: [0,0,0]      # hard-iron [uT], misalignment: soft-iron 3x3
  misalignment: [1,0,0, 0,1,0, 0,0,1]

baro:
  enable: 0                # Process barometer measurements
  stddev_m: 0.0            # Barometer measurement noise [m], 0 -> default
  acc_bias_rw: 0.0         # accel-bias drift [m/s^2/sqrt(Hz)], 0 -> default
  acc_bias_init_mps2: 0.0  # INITIAL accel-bias stddev [m/s^2], 0 -> default
  acc_noise_mps2_sqrthz: 0.0 # accel-noise density [m/s^2/sqrt(Hz)]
  h_process_noise: 0.0     # direct height process noise [m/sqrt(Hz)], 0 -> default
  # local-height/GNSS-ellipsoid offset filter tuning (all 0 -> built-in
  # defaults). Raise local_gnss_rw_stddev_mps further for missions with
  # extreme altitude excursions (e.g. a soaring glider gaining kilometres
  # of altitude): the offset absorbs the ISA-model error, which grows with
  # the excursion and can outrun even the default, leaving the offset
  # permanently lagging.
  local_gnss_rw_stddev_mps: 0.0          # [m/sqrt(s)], 0 -> default 0.3
  local_gnss_chi2_threshold: 0.0         # 0 -> default chi2inv(0.95,1)
  local_gnss_min_update_interval_sec: 0.0 # [s], 0 -> default 10
  local_gnss_stddev_inflation_factor: 0.0 # 0 -> default 3
# Scalar ground speed from speed.csv (odometry, OBD-II). Uncertainty and
# delay are constants here, not per-sample columns.
speed:
  enable: 0                # Process speed.csv
  scale: 0.0               # multiplies the reported speed, 0 -> 1.0
  stddev_mps: 0.0          # per-sample 1-sigma [m/s], 0 -> default
  stddev_rel: 0.0          # speed-proportional 1-sigma, 0 -> default
  min_speed_mps: 0.0       # skip below this FILTERED speed, 0 -> default
  delay_ms: 0              # how old a sample is at its timestamp
# Dual-antenna GNSS heading from heading.csv (moving base NAV-RELPOSNED).
# The 1-sigma is a per-row column, these keys only scale and floor it.

```

```

heading:
  enable: 0 # Process heading.csv
  baseline_frd: [1, 0, 0] # base -> rover antenna, body FRD, any length
  require_fixed: 1 # 1 -> only carrier-phase fixed rows
  stddev_scale: 0.0 # multiplies the row's 1-sigma, 0 -> 1.0
  stddev_min_deg: 0.0 # floor after scaling, 0 -> none
  delay_ms: 0 # how old a row is at its timestamp
# Ranges to anchors at known positions from ranges.csv (REQ-NAV-082).
# The 1-sigma is a per-row column, each row is anchored at its own age.
ranges:
  enable: 0 # Process ranges.csv
  leverarm_frd: [0, 0, 0] # IMU -> ranging antenna, body FRD [m]
  stddev_scale: 0.0 # multiplies the row's 1-sigma, 0 -> 1.0
  stddev_min_m: 0.0 # floor after scaling, 0 -> none
  height_with_baro: 0 # 1 -> ranges also correct the height
  aiding_max_hpos_stddev_m: 0.0 # 0 -> default (REQ-NAV-085)

ahrs:
# SHARED ARS/AHRS noise override (0 -> derived
# from imu: above), both sub-filters consume
# the same physical gyro/accelerometer
  gyr_noise_psd: 0.0 # [rad/s/sqrt(Hz)], 0 -> default
  gyr_bias_rw: 0.0 # [rad/s^2/sqrt(Hz)], 0 -> default
  acc_noise_mps2: 0.0 # [m/s^2], 0 -> default
  gyr_bias_init_stddev_rps_deg: 0.0 # [deg/s], 0 -> unchanged (the
# gyro_bias_window_sec seed's own stddev, or
# ahrs.c's default if that seed never ran)

score:
# scoring/plots + regression gates ONLY (not filter)
  warmup_sec: 60.0
  ahrs: 1 # also score the AHRS/ARS attitude filters
  leverarm_frd: [0,0,0] # map the filter position onto the truth point
  min_epochs: 100
  lim_pos_rms_m: 2.0 # regression guard-rails (make simulated/datasets)
  lim_att_bias_deg: 0.1
  lim_yaw_std_deg: 0.2

inputs:
# optional per-stream CSV filename overrides
# each unset key -> the conventional <stream>.csv
  imu: imu.csv
  ref: ref.csv
  gnss: gnss_f9p.csv # e.g. swap only the receiver, keep the rest shared
  mag: mag.csv
  baro: baro.csv
  speed: speed.csv
  heading: heading.csv
  ranges: ranges.csv

```

Top level.

name *string*. Dataset label used in plots and the summary.

aiding *default gnss*. Absolute-position source: **gnss** uses real per-epoch receiver covariance from **gnss.csv** (zero-diagonal entries replaced by the **gnss**: fallbacks), **ref** synthesises a fix from the ground truth with the **fix**: noise (not an independent error profile), **none** gives no position aiding at all – **ins** never initialises and only the ARS and **baro_alt** filters run (**nav_suite** ATTITUDE-ONLY fallback).

init *default auto*. **auto**: **ins** auto-initialisation. **ref**: known initial state from the truth epoch aligned with the first IMU sample (a moving start is fine, e.g. an aircraft in cruise).

automotive_mode *0/1, default 0*. Derive yaw from GNSS course-over-ground above a minimum speed, typically for wheeled, non-holonomic platforms without a magnetometer. *Not valid* where course differs from heading at speed (aircraft in wind).

automotive_min_speed_mps *m/s* Minimum ground speed for course-derived yaw.

automotive_min_yaw_stddev_deg *deg* Floor on the fused yaw stddev, raise (e.g. 15) to keep it a weak long-term anchor under wind drift.

chi2_disable *0/1, default 0*. Disable χ^2 outlier downweighting library-wide.

chi2_reject_alpha *default 0* Global significance level α of ins's χ^2 downweight gate. Each absolute reference is fused scalar-row-wise (1 DOF), so a configured α sets one shared gate threshold $\chi_{\text{inv}}^2(1 - \alpha, 1)$ for every channel (GNSS, magnetometer, local-position, yaw), evaluated by interpolating a fixed quantile table. $\alpha = 0$ keeps ins.c's own single default (10.0, identical on all four channels – no differing per-channel gates), a larger α (e.g. 0.05 $\rightarrow$ 3.84) is stricter, a smaller one more tolerant (e.g. 0.01 $\rightarrow$ 6.63). Ignored when **chi2_disable** is set.

gyro_bias_window_sec *s* Length of the initial static window averaged to seed the initial gyro-bias estimate.

auto_init_window_sec *s, default 0 $\rightarrow$ default*. Length of the IMU window ins's auto-initialisation levels over (median of the specific force, §5.7). Raise it for a low IMU rate so the window still contains enough samples.

allow_unlimited_deadreckoning, max_deadreckoning_sec *0/1 and s, default 0 $\rightarrow$ default*. IMU-only coasting budget before `is_ready()` degrades (§5.9); the flag makes the window never expire and then overrides the budget.

baro_height_disable *0/1, default 0*. Never select the barometric height source at bootstrap, keep the GNSS position's vertical row, even with a barometer present.

imu: — **noise model (required) and calibration.**

gyr_psd, acc_psd $(\text{rad/s})^2/\text{Hz}$, $(\text{m/s}^2)^2/\text{Hz}$. White-noise PSD (`replay.py` suggests values from a trial). **Required.**

gyr_bias_rw, acc_bias_rw $\text{rad/s}/\sqrt{\text{s}}$, $\text{m/s}^2/\sqrt{\text{s}}$. Bias random walk. Estimate from a long static log with `tools/allan_variance.py`. **Required.**

gyr_fixed_bias, acc_fixed_bias *3-list, rad/s and m/s², default 0*. Removed permanently (`raw - bias`), never estimated.

gyr_misalignment, acc_misalignment *9-list, column-major 3×3 , all-zero $\rightarrow$ identity*. Scale/cross-coupling matrix M , `corrected = M(raw - bias)`. Also feeds the parallel ARS/AHRS.

Measured by `tools/inslib_imu_calib.py` (or the reference board's calibration window) from a session of arbitrary static poses, no fixture involved. The same matrix carries the *housing* alignment when one was captured: the unit is rested on a level surface, any tilt it reports is the IMU sitting crooked in the box, and the rotation that removes it is premultiplied onto M for the accelerometer, the gyroscope and the magnetometer alike. One placement gives roll and pitch; a second on a face that is not parallel to the first also gives the yaw, which gravity on its own can never observe.

mount_rpy_deg *3-list, deg, default 0*. Attitude of the sensor board's axes in the vehicle body frame (ZYX roll, pitch, yaw, FRD), e.g. [0, 0, -9] for a board whose x axis points 9 deg left of the vehicle's forward axis. The harnesses (`tools/replay.c`, `tools/insrcv.c`, `tools/replay.py`, `inspostgui.py`) premultiply the rotation $R(\text{roll}, \text{pitch}, \text{yaw})$, which takes a board-frame vector into the vehicle frame, onto the accelerometer, gyroscope and magnetometer matrices, $M \leftarrow R M$, after the per-axis calibration those already hold. The filter then works in vehicle axes: lever arms, **heading:** `baseline_frd` and every reported attitude refer to the vehicle, not to the board. The configured matrices themselves are left as written (REQ-VER-036).

A yaw mounting error can be read off a drive with a dual-antenna heading: run once with `heading: enable: 0`, so the yaw comes from GNSS velocity and accelerations and therefore follows the board's x axis, and compare it against the heading of the same epochs while driving straight. The median difference is the mounting yaw.

`pos_pred_stddev_m_sqrts`, `vel_pred_stddev_mps_sqrts`, `rpm_pred_stddev_rad_sqrts` $m/\sqrt{s}$, $m/s/\sqrt{s}$, $rad/\sqrt{s}$ Extra process-noise margin added *on top of* the physically-derived acc/gyro-driven pos/vel/rpy prediction noise, not instead of it.

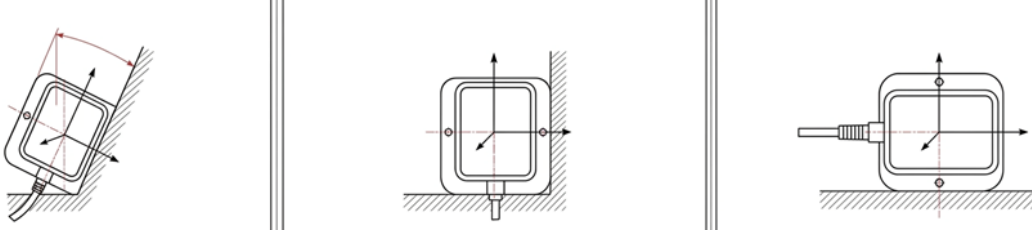


Figure 5: Three of the static poses a calibration session collects: the unit tilted against an edge, resting flat, and standing on its connector end, each read against gravity in the world frame (g , right of each pose). Held-still orientations spanning enough of the sphere are what let the fit separate bias, scale and misalignment without a turntable or fixture – see the coverage sphere below.

The same session also calibrates the magnetometer, when the board streams one (`mag: fixed_bias/misalignment` below): both are fitted from one set of poses, not two separate sessions. Because of that, *the whole session – not only the parts that look magnetometer-specific – has to happen away from ferrous material and active electronics*: a steel desk, a nearby motor, or a metal-legged table all bias the field the magnetometer fit reads, and nothing in the accelerometer/gyroscope channels can detect that it happened. A genuinely magnetically clean surface is harder to find than it sounds.

imu: — stillness (all optional, 0 → a default). One set for the whole suite: it configures `ins`, both AHRS instances and `baro_alt`'s vertical zero-velocity update alike (§5.8). Neither harness configures the ARS/AHRS gates separately any more.

zero_vel_stddev_mps, zero_rot_stddev_deg m/s , deg/s Pseudo-measurement stddev: how hard a detected standstill is trusted once flagged. `zero_vel_stddev_mps` also becomes `baro_alt`'s vertical ZUPT stddev, `zero_rot_stddev_deg` the ARS/AHRS zero-rotation stddev. These two are the one exception to “0 → the library's own default”: both replay harnesses substitute their own 0.05 m/s and 0.1 deg/s, the latter deliberately tighter than `ins`'s built-in 0.5 deg/s, so a config that leaves them at 0 keeps the historical harness tuning instead of silently loosening the rotation channel. Every remaining key below falls back to the library's own default.

auto_zupt_static_gyr_stddev_deg, auto_zupt_static_acc_stddev_mps2 deg/s , m/s^2 The PRIMARY stillness criterion (§5.8): max per-axis RMS sample stddev of the raw gyro/accelerometer over a short window.

auto_zupt_static_gyr_deg, auto_zupt_static_acc_mps2 deg/s , m/s^2 The loose magnitude bounds beside it: $\max \|\omega\|$ and $\max \|f - g\|$ still considered stationary.

auto_zupt_max_vel_mps, auto_zupt_max_vel_stddev_mps m/s The velocity gate: $\max \|v_{GNSS}\|$, and the accuracy (1-sigma, per axis) that GNSS velocity must itself report to count as evidence. Applied only while such a fix is recent enough; with none, only the IMU criteria decide. Not propagated to the ARS/AHRS, which have no velocity state.

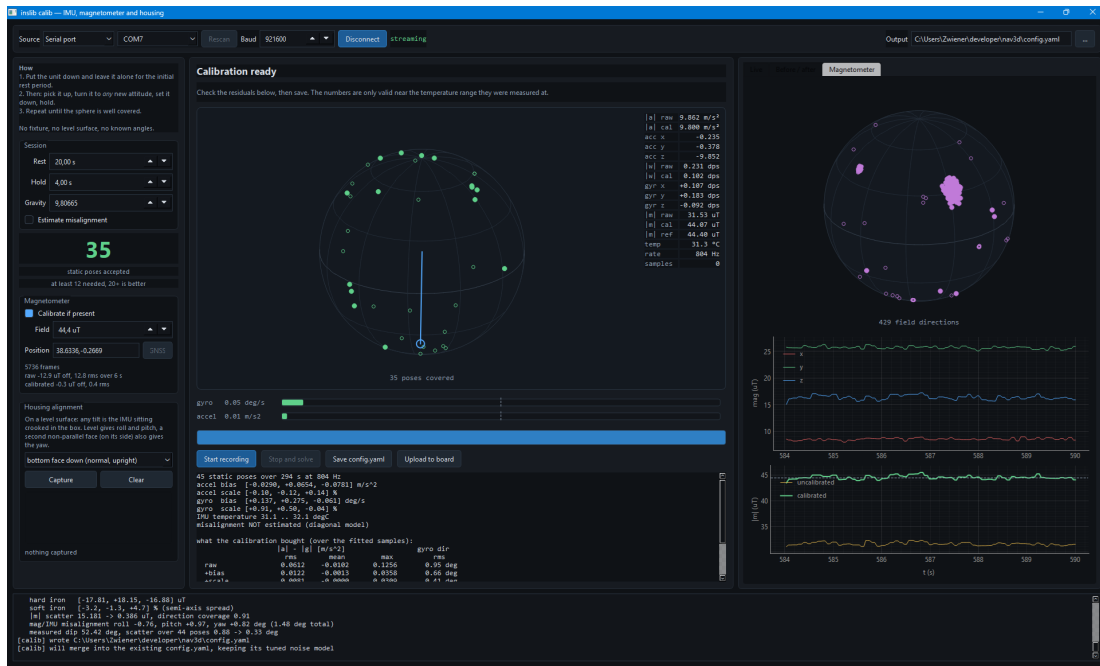


Figure 6: The reference board’s calibration window: the coverage sphere fills in as static poses are accepted, the stillness meter says whether the unit currently held down is calm enough to count, and the live traces show the raw and calibrated readings side by side.

auto_zupt_dwell_sec, **auto_zupt_min_interval_sec** *s* How long the platform must look still before a trigger, and how often triggers may repeat.

auto_zupt_disable *0/1, default 0*. Turn off every stillness source in the suite – **ins**, both AHRS instances and, transitively, **baro_alt**’s vertical ZUPT.

auto_zupt_velocity_blind_disable *0/1, default 0*. Turn off only the ARS/AHRS’s own velocity-blind detector (§5.8); **ins** keeps deciding for all of them.

gnss: — aiding quality (all optional).

leverarm_frd *3-list, m, body FRD, default 0*. IMU-to-antenna lever arm.

delay_ms *ms, default 0*. Assumed fixed processing/telemetry latency, applied uniformly to every fix.

max_{horizontal,vertical}_{pos,vel}_stddev Shutdown gates: a fix whose reported stddev exceeds the limit is ignored.

pos_stddev_fallback_m, **vel_stddev_fallback_mps** [*hor,ver*] *m and m/s, default 0*. Substituted for zero-diagonal reported covariance. **REQUIRED** (no default) under **aiding: ref**: the reference-synthesized fix never has a reported covariance to begin with, so this is the only noise it gets – **load_config()** refuses to run without both.

pos_cov_scale, **vel_cov_scale** *default 0 → 1*. Multiply the reported stddev, for over-optimistic vendor covariance.

pos_cov_scale_height *default 0 → 1*. Extra dedicated downweight of the GNSS position *height* (Down) axis only, for receivers whose vertical solution is far worse than their horizontal one. Applied as the correlation-preserving congruence **SQS** with $\mathbf{S} = \text{diag}(1, 1, s)$ (the N–D and E–D correlation coefficients are kept exactly), on top of **pos_cov_scale**, horizontal accuracy is untouched.

`{pos,vel}_stddev_floor...` *default 0*. Per-axis stddev floor, effective `stddev = max(scale·reported, floor)`.

init_stddev: — **initial-state 1-sigma (REQ-VER-010)**. Each field mirrors `ins_init_t`, 0 falls back to the built-in default. Widen these when the “known” start is itself only a GNSS-derived estimate, or the filter starts overconfident.

`pos_init_stddev_m` m.

`vel_init_stddev_mps` m/s.

`rpy_init_stddev_rad_deg` deg (roll/pitch, and yaw unless the next key is set).

`yaw_init_stddev_rad_deg` deg, 0 $\rightarrow$ `rpy_init_stddev_rad_deg`.

`acc_bias_init_stddev_mps2` m/s².

`gyr_bias_init_stddev_rps_deg` deg/s.

init_hint: — **known initial attitude for auto-init (optional, init: auto only)**. A static “I just know it” hint for `ins`’s auto-init bootstrap (e.g. a known launch heading with no magnetometer/GNSS-course yaw aiding to derive it from). Only used while neither the ARS nor the AHRS has initialised yet (a continuously-running, converged heading takes priority once available) and only until `ins` itself initialises.

`roll_deg, pitch_deg` deg, *default 0*. Known initial roll/pitch, only used together (virtual measurement).

`rpy_stddev_deg` deg, *default 0 $\rightarrow$ no hint*. Shared roll/pitch 1-sigma for the virtual measurement. Usually left unset, since leveling is already observable from gravity.

`yaw_deg` deg, *default 0*. Known initial yaw, independent of roll/pitch. A set yaw hint also seeds the yaw-free ARS’s own initial heading, so `nav_suite_get_rpy()` already reports it during the ATTITUDE_ONLY window instead of the arbitrary 0 the ARS would otherwise start from. Roll/pitch are not seeded there (the ARS levels them from gravity on its first epoch).

`yaw_stddev_deg` deg, *default 0 $\rightarrow$ no hint*. Yaw stays unobservable until aiding arrives otherwise.

ahrs: — **SHARED ARS/AHRS noise override (all optional, default 0)**. The standalone ARS/AHRS attitude filters (§3.3) are physically-separate filter instances from `ins` and are, by default, tuned from the same `imu:` noise model above, **ahrs:** overrides that per dataset instead, one call setting both sub-filters’ template since they consume the same physical gyro/accelerometer.

`gyr_noise_psd` rad/s/ $\sqrt{\text{Hz}}$ Shared gyro noise density.

`gyr_bias_rw` rad/s²/ $\sqrt{\text{Hz}}$, *default 0 $\rightarrow$ default*. Shared gyro bias random walk density.

`acc_noise_mps2` m/s², *default 0 $\rightarrow$ default*. Shared accelerometer noise stddev (leveling).

`gyr_bias_init_stddev_rps_deg` deg/s, *default 0 $\rightarrow$ unchanged*. Initial gyro-bias uncertainty for both sub-filters.

mag: and **baro:** — **optional sensors**.

mag: **enable** *0/1, default 1 live, 0 offline*. Turn on magnetometer heading fusion. The two defaults differ because the key does two different things. Live (`tools/insrcv.c`) the 0x40/0x06 samples arrive over the wire regardless and the key gates the fusion alone (off, they stay decoded, counted and published), so a receiver with no magnetometer simply never fuses one and the default costs nothing — as with the barometer. Offline (`tools/replay.c`, `tools/replay.py`) the same key gates *opening mag.csv* and a missing file is a hard error, so a dataset without a magnetometer stream must not have it on by default. The risk the live default accepts is an *uncalibrated* sensor: hard iron is a bias, and no measurement noise removes the heading error it causes (§5.5.10) — only the calibration below does. It is acceptable because the failure is not silent: once a position is known, the field-strength gate downweights samples whose $|B|$ leaves the tolerance band and warns about a persistent deviation, which is what an uncorrected hard iron of any relevant size looks like. `insrcv` additionally says so at startup when the fusion is on and no calibration is configured.

mag: **stddev_ut** *uT, default 0 → default*. Magnetometer measurement noise. The default is of the same order as the horizontal field itself, i.e. roughly a radian of heading noise per sample, see §5.5.10 for why weak is the right starting point. Lower it only for a sensor whose magnetic environment is known to be clean.

mag: **min_delay_ms** *ms, default 0 → default (1 Hz), negative → no rate limit*. Minimum time between two magnetometer fusions. The second half of the weighting decision above: the noise sets how far one sample may pull the heading, this sets how often it may (§5.5.10). A source that is not a real magnetometer exposed to the platform (a synthesized or externally validated heading stream) is the case for switching the limit off.

mag: **wmm_year** *decimal year*. Epoch for the World Magnetic Model declination lookup. For the live receiver (`insrcv`) it is optional: without it the epoch comes from GPS time in the stream, failing that from the UTC date the receiver reports in NAV-PVT, and failing that from the host clock. The replay harnesses have no date source of their own, so there it is required whenever **mag:** **enable** is set: without it no reference field is built, the magnetometer is not fused, and the harness warns once at start-up. The reference board's capture converter fills it in from the capture's NAV-PVT date.

mag: **estimate_bias** *0/1, default 0*. Run `ins` in its 18-state mode estimating a hard-iron bias online. Observable only under rotation.

mag: **fixed_bias**, **mag:** **misalignment** *3-list uT and 9-list 3×3 , defaults 0/identity*. Fixed hard-iron / soft-iron calibration, $\text{corrected} = M(\text{raw} - \text{bias})$.

Measured alongside the IMU by the same tools when the board streams a magnetometer: turned through every attitude the sensor traces an ellipsoid, whose centre is the hard iron and whose shape is the soft iron. Fitting a sphere cannot see a rotation, so the fit is pinned to the symmetric square root and the leftover rotation is solved separately, from the fact that the angle between the local field and gravity cannot change with attitude. **misalignment** therefore also carries the mounting rotation of the magnetometer onto the IMU triad, without which a magnetometer a few degrees off its board contributes those same few degrees to the fused heading.

baro: **enable** *0/1, default 0 in the replay harnesses, 1 in tools/insrcv.c*. Enable the `baro_alt` vertical channel (needs static pressure measurements). The two defaults differ because the key does two different jobs: offline it gates *opening baro.csv*, which a dataset without a barometer does not have, while the live receiver is handed pressure samples over the wire whether or not anyone asked for them — there a default of 0 would take the vertical channel away from every session whose config predates the key. Set to 0 live, the samples are still decoded, counted, published to the telemetry stream and shown on the stats

line (`baro-off=`), they simply aid nothing, the same A/B-comparable exclusion as `gnss:`
`enable: 0`.

baro: `stddev_m` *m*, *default 0* $\rightarrow$ *default*. Barometric height noise.

baro: `acc_bias_rw` $m/s^2/\sqrt{Hz}$, *default 0* $\rightarrow$ *default*. Continuous-time drift density σ_b of the `baro_alt` vertical-accelerometer bias state ($Q = \sigma_b^2 \Delta t$), larger values let the estimated bias track faster. Note that this is a bias in the local navigation frame, not in the body frame.

baro: `acc_bias_init_mps2` m/s^2 , *default 0* $\rightarrow$ *default*. `baro_alt`'s own INITIAL accel-bias uncertainty – an initial condition, not a process-noise rate.

baro: `acc_noise_mps2_sqrthz` $m/s^2/\sqrt{Hz}$, *default 0* $\rightarrow$ *default*. `baro_alt`'s own direct accelerometer-noise density, separate from other accelerometer noise settings as the measurement here is transformed into the navigation frame and can, for example, include attitude uncertainty.

baro: `h_process_noise` $m/\sqrt{Hz}$, *default 0* $\rightarrow$ *default*. Small direct process noise density σ_h on `baro_alt`'s height state (`h_process_noise_m_sqrthz`, §7.2) – a safety margin against discretization/model mismatch, not a primary noise source.

heading: — **dual-antenna GNSS heading (optional, REQ-VER-035).**

heading: `enable` *0/1*, *default 0*. Open `heading.csv` and fuse its rows as absolute yaw (§5.5.9), the newest row of each IMU interval. A missing file is a hard error, as for `mag/baro/speed`.

heading: `baseline_frd` *3-list*, *default [1, 0, 0]*. Direction from the moving base antenna to the rover antenna in the body frame, any length. The file holds the azimuth of that vector as measured, so a remounted antenna pair is a config edit, not a reconversion.

heading: `require_fixed` *0/1*, *default 1*. Skip rows whose carrier-phase solution is not fixed. A float baseline can be decimetres off, which across a metre of antenna separation is degrees of heading while the receiver's accuracy still claims less.

heading: `stddev_scale`, **heading:** `stddev_min_deg` *default 0* $\rightarrow$ *1.0 / no floor*. The row's 1-sigma (the receiver's `accHeading`) is multiplied by the scale and then floored. The formal accuracy knows nothing about antenna phase-centre variation or a flexing baseline, which is what the floor is for.

heading: `delay_ms` *ms*, *default 0*. Age of a row at its timestamp, anchored in the state history like `gnss: delay_ms`. The converter stamps a heading with the time of the NAV-PVT of the same epoch, so the GNSS delay is the starting point.

Each harness prints how many rows were offered as yaw and how many were skipped as not fixed, for a bad 1-sigma, or by the baseline geometry.

ranges: — **ranges to anchors (optional, REQ-VER-038).** `ranges.csv` holds one calibrated range per row: `t_us` (the time the range refers to), the anchor id, the anchor position in ECEF, the range and its 1-sigma. Each producer does its own calibration, the harnesses only weight the rows.

ranges: `enable` *0/1*, *default 0*. Open `ranges.csv` and fuse every row up to each IMU epoch as a range measurement (REQ-NAV-082), anchored in the state history at its own age. Rows beyond the filter's per-epoch maximum move on to the next epoch. A missing file is a hard error.

ranges: `leverarm_frd` 3-list, default [0,0,0]. From the IMU to the ranging antenna, body frame.

ranges: `stddev_scale`, **ranges:** `stddev_min_m` default 0 $\rightarrow$ 1.0 / no floor. The row's 1-sigma is multiplied by the scale and then floored.

ranges: `height_with_baro`, **ranges:** `aiding_max_hpos_stddev_m` default 0. The `ins_options_t` keys of the same names: ranges also correct the height under the barometric height source, and the horizontal 1-sigma below which fused ranges count as position aiding for the coasting window (REQ-NAV-085).

Each harness, `inspostgui.py` included, prints how many rows were offered, fused, rejected by the chi2 gate or skipped, and in how many epochs the ranges counted as position aiding. To score a ranges-only stretch of a recording, cut the GNSS fixes of that stretch with `replay.py --gnss-outage` (§11.4) or the GUI's *GNSS outage* field, or cut the rows from a copy of `gnss.csv` and hand it over with `inputs: gnss`. The reference stays the full track either way.

Non-holonomic lateral constraint (top level, optional). `automotive_lateral_constraint` [0/1, default 0] fuses a synthetic measurement stating that a wheeled vehicle does not travel sideways: the body-frame lateral velocity against a truth of zero (REQ-NAV-077). It needs `automotive_mode` and shares its `automotive_min_speed_mps`. What the constraint is for is its attitude row, whose gain is the ground speed: at 20 m/s one degree of roll error is 0.35 m/s of lateral body velocity, and during a GNSS outage that channel is otherwise unobserved. `automotive_lateral_stddev_mps` [m/s] is the measurement noise, `automotive_lateral_max_yaw_rate_deg` [deg/s] skips the fusion in turns where side slip breaks the assumption, and `automotive_lateral_after_sec` [s] holds it off until the filter has been that long without a GNSS fusion.

score: — evaluation only (never touches the filter). Used by the accuracy summary, the plots and the regression gates. `warmup_sec` [s, default 60] delays scoring after the first fix, `ahrs` [0/1] also scores the attitude filters, `leverarm_frd` [m] maps the filter position onto the ground-truth point (e.g. the antenna, when `ref.csv` is the receiver's own solution): the position error, the ellipsoid height error and every estimate the plots overlay on the reference (state history, North-East map, both altitude pages, where the barometer's curves move with it) refer to that point, so a lever arm is never reported as filter error (REQ-VER-037). `min_epochs` sets the minimum scored-epoch count. The `lim_*` keys are the per-dataset guard-rails that make `simulated/make datasets` trip on, they are tuned regression thresholds, not accuracy claims, and each one at 0 is simply not gated: `lim_pos_rms_m` on horizontal position, `lim_att_bias_deg/lim_att_std_deg` and `lim_yaw_bias_deg/lim_yaw_std_deg` on attitude, `lim_baro_{rms,bias,max}_m` on the `baro_alt` *relative* height, `lim_ellipsoid_{rms,bias,max}_m` on the *absolute* ellipsoidal height the suite reports (§7.1), and the `lim_ars_*` attitude-fallback limits (`lim_ars_att_bias_deg`, `lim_ars_roll_std_deg`, `lim_ars_pitch_std_deg`, `lim_ars_yaw_drift_deg_min`), which are only checked with `ahrs: 1`. `attitude` [0/1, default 1] is the opt-out for a dataset whose `ref.csv` carries no attitude truth: with it at 0 the attitude columns are neither scored nor gated.

`ref_delay_ms` [ms, default 0] states that a reference row timestamped T describes the state at T minus that delay, and the harness scores every comparison at that time of validity (REQ-VER-030). Set it to mirror `gnss: delay_ms` whenever `ref.csv` comes out of the same receiver output as the aiding: `gnss: delay_ms` tells the filter how old a fix is so that it reports where the vehicle is NOW, while the reference row still describes where it was, so comparing both at a shared timestamp penalises the filter by speed times delay — and penalises it more the better it compensates. Leave it at 0 for an independent reference (Lighthouse, synthetic

truth) and for a dataset that models the latency in its data rather than its config. A nonzero value with `aiding: ref` is refused, since there the reference is the aiding.

`coast_gap_min_sec` [s, 0 o off] turns on the coasting re-acquisition metric (REQ-VER-029): every gap in the aiding file longer than it is reported with its duration, the straight-line distance across it and the filter's position error at the first IMU epoch at or after its end, scored against the reference epoch nearest to that IMU epoch (none further than half a second, or half the gap, away), and `lim_coast_exit_err_m` [m, 0 o not gated] is the guard-rail on that error. This is the metric for a dataset with a real outage, where the whole-run `lim_pos_rms_m` cannot answer the question: no fixes means no reference inside the gap, and a filter that gives the coasting solution up scores BETTER on the RMS than one that coasts through, because it contributes no epochs where the error is large. The error is sampled before that epoch's measurements are fused — the returning fix snaps the position onto the truth in the same epoch it arrives, so a sample taken afterwards reports how good the fix was rather than how far the coasting drifted. Having no solution when aiding returns counts as a failure of this gate rather than as a skipped epoch. The gaps are taken from the aiding file, not from what the filter chose to fuse, so the number stays comparable across configurations that gate fixes differently.

inputs: — **CSV filename overrides (optional).** Per-stream keys `imu`, `ref`, `gnss`, `mag`, `baro`, each a filename resolved relative to the `config.yaml` directory. An unset key keeps the conventional `<stream>.csv` name, so a dataset without an `inputs:` block behaves exactly as before. This lets one stream be swapped (e.g. `gnss: gnss_f9p.csv`) while the remaining CSVs stay shared — an A/B comparison of two receivers or IMUs needs no directory duplication.

11.1 Parameter cross-reference: `config.yaml` ↔ C API

Table 1 lists every `config.yaml` key next to the C struct field or function call that ultimately consumes it, derived from `tools/replay.c`'s `cfg_set()` (the canonical, exhaustive mapping, REQ-VER-025) — `tools/replay.py`'s `build_config()` wires the identical schema onto the same field names via ctypes. A key with no C equivalent is consumed by the harness itself (dataset selection, CSV file names, scoring) and never reaches the library. Fields written per epoch (into `ins_measurements_t`, see §11) are marked (*per epoch*); everything else is set once, before the replay loop starts.

Table 1: config.yaml key → C field / call.

config.yaml key	C field / call (header)
Top level	
name	harness-only (dataset label)
aiding	harness-only (selects ins aiding path)
init	harness-only (selects ins_options_t.auto_init)
automotive_mode	ins_options_t.automotive_mode (ins.h)
automotive_min_speed_mps	ins_options_t.automotive_min_speed_mps (ins.h)
automotive_min_yaw_stddev_deg	ins_options_t.automotive_min_yaw_stddev (ins.h)
chi2_disable	ins_options_t.chi2_disable (ins.h)
chi2_reject_alpha	ins_options_t.chi2_reject_alpha (ins.h)
allow_unlimited_deadreckoning	ins_options_t.allow_unlimited_deadreckoning (ins.h)
max_deadreckoning_sec	ins_options_t.max_deadreckoning_sec (ins.h)
baro_height_disable	ins_options_t.baro_height_disable (ins.h)
gyro_bias_window_sec	harness-only heuristic (seeds ins_init_t.gyr_bias_init_rps / ahrs_config_t.gyr_bias_init_rps)
auto_init_window_sec	ins_options_t.auto_init_window_sec (ins.h)
imu: — noise model and calibration	
gyr_psd	ins_measurements_t.gyr.Qll_diag[] (<i>per epoch</i>) (ins.h)
acc_psd	ins_measurements_t.acc.Qll_diag[] (<i>per epoch</i>) (ins.h)
gyr_bias_rw	ins_init_t.gyr_bias_pred_stddev_rps_sqrts (ins.h); also the ahrs: default (ahrs_config_t.gyr_bias_rw, ahrs.h)
acc_bias_rw	ins_init_t.acc_bias_pred_stddev_mps2_sqrts (ins.h)
gyr_fixed_bias	ins_options_t.imu_gyr_fixed_bias[3] (ins.h)
acc_fixed_bias	ins_options_t.imu_acc_fixed_bias[3] (ins.h)
gyr_misalignment	ins_options_t.imu_gyr_misalignment[9] (ins.h)
acc_misalignment	ins_options_t.imu_acc_misalignment[9] (ins.h)
pos_pred_stddev_m_sqrts	ins_init_t.pos_pred_stddev_m_sqrts (ins.h)
vel_pred_stddev_mps_sqrts	ins_init_t.vel_pred_stddev_mps_sqrts (ins.h)
rpy_pred_stddev_rad_sqrts	ins_init_t.rpy_pred_stddev_rad_sqrts (ins.h)
zero_vel_stddev_mps	ins_init_t.zero_vel_stddev_mps (ins.h)
zero_rot_stddev_deg	ins_init_t.zero_rot_stddev_rps (ins.h)
auto_zupt_static_gyr_stddev_deg	ins_options_t.auto_zupt_static_gyr_stddev_rps (ins.h)
auto_zupt_static_acc_stddev_mps2	ins_options_t.auto_zupt_static_acc_stddev_mps2 (ins.h)
auto_zupt_static_gyr_deg	ins_options_t.auto_zupt_static_gyr_rps (ins.h)

(continued on next page)

(continued from previous page)

config.yaml key	C field / call (header)
auto_zupt_static_acc_mps2	ins_options_t.auto_zupt_static_acc_mps2 (ins.h)
auto_zupt_max_vel_mps	ins_options_t.auto_zupt_max_vel_mps (ins.h)
auto_zupt_max_vel_stddev_mps	ins_options_t.auto_zupt_max_vel_stddev_mps (ins.h)
auto_zupt_dwell_sec	ins_options_t.auto_zupt_dwell_sec (ins.h)
auto_zupt_min_interval_sec	ins_options_t.auto_zupt_min_interval_sec (ins.h)
auto_zupt_disable	ins_options_t.auto_zupt_disable (ins.h)
auto_zupt_velocity_blind_disable	ins_options_t.auto_zupt_velocity_blind_disable (ins.h)
gnss: — aiding quality	
leverarm_frd	ins_measurements_t.gnss_leverarm_b[3] (<i>per epoch</i>) (ins.h)
delay_ms	ins_measurements_t.gnss_delay_ms (<i>per epoch</i>) (ins.h)
pos_stddev_fallback_m	harness-only: fills a zero-diagonal ins_meas_gnss_pos_t.Qll_ned (<i>per epoch</i>)
vel_stddev_fallback_mps	harness-only: fills a zero-diagonal ins_meas_gnss_vel_t.Qll_ned (<i>per epoch</i>)
max_horizontal_pos_stddev_m	ins_options_t.gnss_max_horizontal_pos_stddev_m (ins.h)
max_vertical_pos_stddev_m	ins_options_t.gnss_max_vertical_pos_stddev_m (ins.h)
max_horizontal_vel_stddev_mps	ins_options_t.gnss_max_horizontal_vel_stddev_mps (ins.h)
max_vertical_vel_stddev_mps	ins_options_t.gnss_max_vertical_vel_stddev_mps (ins.h)
start_max_horizontal_pos_stddev_m	ins_options_t.gnss_start_max_horizontal_pos_stddev_m (ins.h)
start_max_vertical_pos_stddev_m	ins_options_t.gnss_start_max_vertical_pos_stddev_m (ins.h)
start_max_horizontal_vel_stddev_mps	ins_options_t.gnss_start_max_horizontal_vel_stddev_mps (ins.h)
start_max_vertical_vel_stddev_mps	ins_options_t.gnss_start_max_vertical_vel_stddev_mps (ins.h)
stop_max_horizontal_pos_stddev_m	ins_options_t.gnss_stop_max_horizontal_pos_stddev_m (ins.h)
stop_max_vertical_pos_stddev_m	ins_options_t.gnss_stop_max_vertical_pos_stddev_m (ins.h)
stop_max_horizontal_vel_stddev_mps	ins_options_t.gnss_stop_max_horizontal_vel_stddev_mps (ins.h)
stop_max_vertical_vel_stddev_mps	ins_options_t.gnss_stop_max_vertical_vel_stddev_mps (ins.h)
init_dwell_sec	ins_options_t.gnss_init_dwell_sec (ins.h)
init_dwell_disable	ins_options_t.gnss_init_dwell_disable (ins.h)
stop_dwell_sec	ins_options_t.gnss_stop_dwell_sec (ins.h)
stop_disable	ins_options_t.gnss_stop_disable (ins.h)
pos_decimation	ins_options_t.gnss_pos_decimation (ins.h)
pos_cov_scale	ins_options_t.gnss_pos_cov_scale (ins.h)
pos_cov_scale_height	ins_options_t.gnss_pos_cov_scale_height (ins.h)
vel_cov_scale	ins_options_t.gnss_vel_cov_scale (ins.h)
pos_stddev_floor_hor_m	ins_options_t.gnss_pos_stddev_floor_hor_m (ins.h)

(continued on next page)

(continued from previous page)

config.yaml key	C field / call (header)
pos_stddev_floor_ver_m	ins_options_t.gnss_pos_stddev_floor_ver_m (ins.h)
vel_stddev_floor_hor_mps	ins_options_t.gnss_vel_stddev_floor_hor_mps (ins.h)
vel_stddev_floor_ver_mps	ins_options_t.gnss_vel_stddev_floor_ver_mps (ins.h)
init_stddev: — initial-state 1-sigma	
pos_init_stddev_m	ins_init_t.pos_init_stddev_m (ins.h)
vel_init_stddev_mps	ins_init_t.vel_init_stddev_mps (ins.h)
rpy_init_stddev_rad_deg	ins_init_t.rpy_init_stddev_rad[0..1] (ins.h)
yaw_init_stddev_rad_deg	ins_init_t.rpy_init_stddev_rad[2] (ins.h)
acc_bias_init_stddev_mps2	ins_init_t.acc_bias_init_stddev_mps2 (ins.h)
gyr_bias_init_stddev_rps_deg	ins_init_t.gyr_bias_init_stddev_rps (ins.h)
init_hint: — auto-init attitude hint	
roll_deg, pitch_deg, rpy_stddev_deg, yaw_deg, yaw_stddev_deg	arguments of nav_suite_set_init_att_hint() (nav_suite.h)
mag: — magnetometer	
enable	harness-only (gates reading mag.csv / setting ins_measurements_t.mag.is_valid)
stddev_ut	ins_measurements_t.mag.Qll_diag[] (<i>per epoch, squared</i>) (ins.h)
min_delay_ms	ins_options_t.magnetometer_min_delay_ms (ins.h)
wmm_year	argument of ins_set_magnetic_model_from_position() (ins.h) and ahrs_set_position() (ahrs.h)
estimate_bias	ins_options_t.estimate_mag_bias (ins.h)
fixed_bias	ins_options_t.mag_fixed_bias[3] (ins.h)
misalignment	ins_options_t.mag_misalignment[9] (ins.h)
baro: — barometer	
enable	consumer-only (gates reading baro.csv resp. forwarding a received sample / setting ins_measurements_t.bar.is_valid)
stddev_m	ins_measurements_t.bar.stddev_m (<i>per epoch</i>) (ins.h, also consumed by baro_alt via nav_suite)
acc_bias_rw	baro_alt_config_t.acc_bias_drift_mps2_sqrthz (baro_alt.h)
acc_bias_init_mps2	baro_alt_config_t.acc_bias_init_stddev_mps2 (baro_alt.h)
acc_noise_mps2_sqrthz	baro_alt_config_t.acc_noise_mps2_sqrthz (baro_alt.h)
h_process_noise	baro_alt_config_t.h_process_noise_m_sqrthz (baro_alt.h)
ahrs: — shared ARS/AHRS noise override	
gyr_noise_psd	ahrs_config_t.gyr_noise_psd (ahrs.h), applies to both nav_suite_t.ars_cfg and .ahrs_cfg

(continued on next page)

(continued from previous page)

config.yaml key	C field / call (header)
gyr_bias_rw	ahrs_config_t.gyr_bias_rw (ahrs.h)
acc_noise_mps2	ahrs_config_t.acc_noise_mps2 (ahrs.h)
gyr_bias_init_stddev_rps_deg	ahrs_config_t.gyr_bias_init_stddev_rps[3] (ahrs.h)
score: and inputs: — evaluation only	
score:*	harness-only (scoring, plots, regression gates – never reaches the filter)
inputs:*	harness-only (per-stream CSV filename overrides)

11.2 Free-inertial mode (`free_inertial_start`)

The 3D solution normally begins with a GNSS fix, because the strapdown algorithm needs an absolute position to integrate away from. Free-inertial mode allows it to manually supply the origin (instead of a GNSS receiver).

How it starts. The declared position is offered to the filter as a position measurement – with the uncertainty the operator states, and subject to the ordinary 3D entry gate – and *only until the filter has bootstrapped from it*. Then the source goes silent. Once the bootstrap has happened the filter runs on the IMU, the auto-ZUPTs (section 5.8) and the barometer, which is selected as the height source at that bootstrap like in any other run (REQ-NAV-053).

What it deliberately switches off. Enabling the mode is a statement that coasting is the intent, not a fault, so the two mechanisms that would otherwise end the run are taken out of the loop: the dead-reckoning budget (`max_deadreckoning_sec`) and the 3D exit gate (`gnss.stop_disable`, REQ-NAV-052). Both exist to protect an *aided* solution from quietly degrading, and both would stop the filter mid-experiment for doing exactly what it was asked to do. The 3D entry gate stays in force and every fix is still judged on its own accuracy before it may fuse: GNSS keeps its power to *correct* the solution, it only loses its power to *end* it.

What it cannot deliver. Heading is unobservable: without a GNSS course and without a magnetometer nothing measures yaw, and at standstill a MEMS gyroscope cannot find north either, its bias being orders of magnitude above the earth rate. The dead-reckoned track therefore has a shape but no absolute direction, and the yaw variance grows to “unknown” – unless the operator states the start heading themselves via `init_hint` (section 11), e.g. a bench test with the unit facing a known direction, in which case that stated uncertainty is what the yaw variance starts from instead. Horizontal position drifts with nothing on board to correct it. The barometer holds the vertical channel and the ZUPTs hold velocity and the IMU biases whenever the platform stands still – that is what makes such a run informative, and it is also all of it.

A typical configuration for an indoor inertial-sensor run, GNSS excluded outright (shipped as `doc/example_conf/config_local_inertial.yaml`, a starting point for a quick test of pure inertial navigation):

```
aiding: none                                # replay only: no gnss.csv needed (insrcv
                                           # ignores the key, gnss: enable 0 below is
                                           # its counterpart there)

chi2_disable: 1                             # outlier rejection off: expected at a standstill
                                           # bootstrap and during ZUPT-only coasting

free_inertial_start:                       # implemented by insrcv, both replay harnesses
                                           # and inspostgui
  enable: 1                                # implies: no DR budget, no GNSS-quality exit
  lat_deg: 49.0                            # where the platform starts, as well as known
  lon_deg: 8.4
  height_m: 118.0
  stddev_m: 0.05                           # 1-sigma of that statement; must pass the
                                           # 3D entry gate (gnss: start_max_horizontal_
                                           # pos_stddev_m), so keep it at or below it

init_hint:                                # known start heading, e.g. a bench test with
                                           # the unit facing a defined/known direction
  yaw_deg: 0.0
  yaw_stddev_deg: 0.5                      # 0 -> no hint (usual: stays unknown, see
                                           # above); >0 -> use it

init_stddev:
```

```

pos_init_stddev_m: 5.00
vel_init_stddev_mps: 0.05
yaw_init_stddev_rad_deg: 0.1 # unused while init_hint.yaw_stddev_deg > 0

gnss:
  enable: 0 # decode and publish, but aid nothing with it

baro:
  enable: 0 # off here: this example is pure IMU-only
  acc_bias_init_mps2: 0.05
  stddev_m: 2.0

mag:
  enable: 0

imu: # the measured noise model of THIS sensor -
  gyr_psd: 5.0e-07 # in this mode nothing else constrains the
  acc_psd: 1.0e-05 # solution, so it is what the run is about
  gyr_bias_rw: 1.5e-05
  acc_bias_rw: 3.0e-04
  zero_vel_stddev_mps: 0.05 # the ZUPT is the only velocity measurement
  auto_zupt_dwell_sec: 0.1
  auto_zupt_static_gyr_stddev_deg: 0.35 # stillness thresholds against
  auto_zupt_static_acc_stddev_mps2: 0.45 # the sensor's own noise
  zero_rot_stddev_deg: 0.03 # left, so its stddev matters more than usual

```

This example also excludes the barometer. With `baro: enable: 0` the vertical channel runs on the accelerometer and the ZUPTs alone, drifting like the horizontal solution – deliberately, to demonstrate the pure-IMU extreme (no GNSS, no barometer, no magnetometer). A barometer, where fused, would hold the vertical channel through the same outage that leaves the horizontal solution to drift, exactly as described above.

Excluding GNSS outright. `gnss: enable: 0` is a key of its own, usable with or without the free-inertial start and implemented by all three consumers. The fixes are still decoded, counted and (live) published to the telemetry stream – they simply aid nothing. That is deliberately different from deleting `gnss.csv` or setting every gate to reject: the run stays directly comparable against the same recording processed *with* GNSS, one key apart, which is what makes it an A/B rather than two different experiments. On a real dataset the difference reads straight off the summary – the same 3073 GNSS fixes are reported either way, while the filter goes from 2950 GNSS fusions to none.

With GNSS excluded the magnetic model takes its anchor from the declared position rather than from a fix, since declination and the reference field vary on the scale of degrees of latitude and a hand-entered position is ample for them. Its epoch comes from `mag: wmm_year` if the config states one, from GPS time when the stream carries a pulse, from the UTC date of the receiver's own NAV-PVT when it does not, and otherwise from the host clock: a date is a clock reading rather than a position measurement, so excluding GNSS as an aiding source does not exclude it as a calendar, and no epoch at all would mean no model at all, i.e. an AHRS referencing magnetic north without saying so. The live receiver names the source it used in its log line, so a wrong date is traceable rather than merely suspected.

The same run offline. All three consumers implement the section: the live receiver `tools/insrcv.c` and both replay harnesses (`tools/replay.c`, `tools/replay.py`). The same `config.yaml` therefore produces the same start whether the data arrives over UDP or out of `imu.csv`. The offer mechanism is identical in all three – the declared position at 2 Hz until `ins` has bootstrapped from it, never in an epoch that already carries a real fix, and the same two forced options – which is the point of a shared schema: a key that did something live and was quietly ignored on replay would be precisely the silently ineffective setting this chapter argues against.

Replayed this way a dataset needs no `gnss.csv` at all (`aiding: none`), the declaration does the job the first fix would otherwise do. For a stationary run the reference is worth having on top: if the platform does not move, the truth is that it does not move, so a `ref.csv` holding the constant start position turns the harness's scored position error into a direct readout of the free-inertial drift.

```
name: indoor-free-inertial
aiding: none           # no gnss.csv, the declaration starts the filter
init: auto             # bootstrap from it, exactly as the live path does
```

The same trick works for a run that moves in between, as long as it returns to where – and, if the mount does not turn freely, how – it started: `datasets/inertial_roundtrip` carries a hand-carried, 160.7s pure-inertial recording whose `ref.csv` holds only the start and end pose, identical because the unit was placed back on the same physical mark. `score: warmup_sec` is set to fall shortly before the end epoch's timestamp so the start row (the auto-init anchor) is excluded and only the end row is ever scored, turning the round-trip position AND yaw error into a tight regression gate with no ground truth needed for the 160s in between (REQ-VER-034).

11.3 Dataset CSV streams

Alongside `config.yaml`, a dataset directory carries the sensor and ground-truth streams as plain comma-separated files in one dataset-neutral format (REQ-VER-003), written by the shared helpers in `datasets/replay_format.py` so the converters cannot drift apart and both harnesses read the same bytes. Only `imu.csv` drives the filter, `ref.csv` is the independent ground truth (used for scoring, for `init: ref`, and to synthesise fixes under `aiding: ref`), `gnss.csv`, `mag.csv`, `baro.csv`, `speed.csv` and `heading.csv` are consumed only when the matching `aiding/sensor` option is on. The first line of each file is a `#` header, every column layout:

```
# imu.csv      (required, drives the filter)
t_us, gx, gy, gz [rad/s], ax, ay, az [m/s^2]
t_us, gx, gy, gz [rad/s], ax, ay, az [m/s^2], temp_c    (optional column)

# ref.csv      (required -- ground truth, ZYX roll/pitch/yaw)
t_us, lat_deg, lon_deg, h_m, roll_deg, pitch_deg, yaw_deg, vn, ve, vd [m/s]

# gnss.csv     (aiding: gnss)    -- 20 fields, one fix per row
t_us, lat_deg, lon_deg, h_m,
      cov_pos_ned nn,ne,nd,ee,ed,dd [m^2],
      vn, ve, vd [m/s],
      cov_vel_ned nn,ne,nd,ee,ed,dd [(m/s)^2],
      vel_ok

# mag.csv      (optional, mag: enable)
t_us, mx, my, mz [uT]

# baro.csv     (optional, baro: enable)
t_us, static_pressure [Pa]

# speed.csv    (optional, speed: enable) -- any further columns are the
#              producer's own record, both harnesses read the first two
t_us, speed_mps

# heading.csv  (optional, heading: enable) -- baseline azimuth, NOT yet
#              the yaw, carr_soln 0 none / 1 float / 2 fixed. Same t_us as
#              the gnss.csv fix of the same epoch. Both harnesses read the
#              first four columns, the rest is the producer's record
t_us, heading_deg, stddev_deg, carr_soln, length_m, itow_ms
```

Conventions shared by all streams:

Timestamp `t_us` `int64` microseconds on one common clock, monotonically non-decreasing, the harness feeds each IMU epoch and then the most recent sample of every other stream at or before that time.

Frames Body **FRD** (x forward, y right, z down) for IMU/mag, navigation **NED** for velocity and covariances, angles in degrees (**ref** attitude is Tait–Bryan ZYX roll/pitch/yaw). Position is geodetic `lat_deg/lon_deg` with ellipsoid height `h_m`.

IMU `gx..gz` are the body angular rate [rad/s], `ax..az` the specific force [m/s²] (gravity included, so a level static unit reads $\approx (0, 0, -g)$).

GNSS covariance The 6 unique elements of the symmetric 3×3 NED covariance in order `nn,ne,nd,ee,ed,dd`. A *zero diagonal* entry means “unknown” and is replaced at replay time by the `gnss: fallback stddev` (§11), `vel_ok` (0/1) flags whether the velocity data is usable.

Header/comment lines Any line beginning with `#` is skipped, so the header row is informational. Rows with fewer than the required number of columns are ignored.

11.4 replay.py analysis options (command line)

The *filter* behaviour is fixed by `config.yaml` (§11), the Python harness `tools/replay.py` adds a few command-line flags that only affect *analysis and output*, never the estimate itself, plus `--gnss-outage`, which takes data away from the filter. `tools/replay.py --help` is authoritative, the flags that change what is evaluated or reported are:

--gnss-outage `START:DURATION` Simulate a GNSS outage: every fix from `START` to `START+DURATION` seconds after the first IMU sample (the origin of `--eval-start`) is cut from the input before the run, so the filter never learns that a fix existed. Repeatable, `aiding: gnss` only. Combine it with `--eval-start/--eval-end` to score the drift inside the gap rather than diluting it over the whole run. `inspostgui.py` offers the same cut as its *GNSS outage* toolbar field (comma-separated windows) and as `--gnss-outage` with `--batch`.

--eval-start `SEC`, **--eval-end** `SEC` Restrict the ground-truth error evaluation to a trial-time window (seconds from the first IMU sample, the same origin as the printed `t=...` times). The filter still runs over the *whole* trial – so it stays converged – only the scored attitude/position error statistics and the `--summary-json` output are limited to the window. The sensor data-quality checks (standstill accuracy, IMU noise) still use the full trial. Useful to exclude a bad startup segment or focus on one leg. Either bound may be given alone, the active window is echoed in the “*ins vs ground truth*” header.

--estimate-gnss-delay Force the GNSS-delay cross-correlation estimate on even outside the auto-detected case (`aiding: gnss` with usable velocity plus a barometer already runs it automatically, REQ-VER-009). Prints the estimated delay, its correlation quality, and – when the correlation is trustworthy – an actionable hint to set config `gnss: delay_ms` (§11) to that value, the same number and hint appear on the GNSS-delay page of the `--plot` PDF. The estimate is *relative to baro_alt*’s own (not necessarily zero) group delay.

--summary-json `PATH` Write the end-of-run accuracy summary (scored-epoch count, position RMS and maximum, roll/pitch/yaw error mean/std/max, warmup and evaluation window) as machine-readable JSON. Consumed by the simulated-dataset regression harness (`datasets/check_simulated.py`, REQ-VER-016). It carries no pass/fail gates of its own (this stays a pure analysis tool).

--plot, --plot-out PATH Render the multi-page diagnostic figures (state history, error, bias, ARS/AHRS, `baro_alt`, altitude, GNSS delay, sensor rate, outliers) interactively or, with **--plot-out**, as a single PDF (implies **--plot**).

--plot-hz HZ, --plot-track-hz HZ Sampling rates of the two **--plot** recorders. **--plot-hz** drives the state history behind every time-series page (roughly 60 fields per tick). **--plot-track-hz** drives the North-East map page alone, which stores nothing but the North/East pair and therefore runs an order of magnitude faster: on a track spanning a few metres, every sample the recorder misses shows up as a chord across the real path. The map page then thins its vertices geometrically, to a tolerance relative to the extent of the track rather than to a fixed sample count, so the drawn resolution follows the scale of the view (millimetres on a 2m track, metres on a 10 km one) at nearly constant page size. 0 makes the map reuse the **--plot-hz** samples.

`replay.py` and `inspostgui.py` run the same replay core (`tools/replay_core.py`): input loading, feed order, scoring, recorders and the printed summary are one implementation, so the GUI and the command line give the same result for the same `config.yaml`. `python/tests/test_replay_core.py` checks that on a dataset with ranges and a GNSS outage.

12 Verification environment (overview)

For completeness – the process itself is documented in `requirements/README.md`:

- **Requirements database** (`requirements/req_*.md`): aerospace-style, machine-checked by `make reqs` (unique IDs, status, parent links, and resolution of every `Test`: reference against the actual test code, `@satisfies` trace tags in `src/`).
- **Synthetic tests** (`make test`): plain-C scenario tests for all filters, no framework, non-zero exit on failure.
- **Real-data replays** (`make datasets`): dataset processing harness (`tools/replay.c`, mirrored by `tools/replay.py`) replays datasets for verification.
- **Static analysis** (`make cppcheck`, `make clang-tidy`): complements the compiler warnings (`-Wall -Wextra` plus a wider set) with two independent tools over the production sources.
- **Stack usage** (`make stack`): compiles the library with GCC's `-fcallgraph-info=su`, which reports the frame size of every function and the call graph after inlining, and computes the worst-case stack of every public API function along its deepest call chain (`scripts/stack_usage.py`). It fails on recursion, variable length arrays, calls through function pointers or into library functions that `scripts/stack_usage.cfg` does not give a figure for, and on a worst case above its budget in that file. The budgets there are for an `x86_64` host. For a target, run the script with its cross compiler and build flags and a config holding the figures of its C library: the script reads several config files and, given the stack a program reserves, checks the main task plus nested interrupt handlers and exception frames against it. Frame sizes depend on the architecture and the optimization level, so a host figure is no substitute for the target's. `make check-all` runs every gate above (plus `make coverage` and a `make doxygen` pass that treats any undocumented function or parameter as a failure) independently and prints a pass/fail summary, for pre-push or CI use.
- **Coverage** (`make coverage`): lcov HTML report including branch (and, with `gcc ≥ 14.2`, MC/DC) coverage.

References

- [1] J. Wendel, *Integrierte Navigationssysteme: Sensordatenfusion, GPS und Inertiale Navigation*, 2nd ed., Oldenbourg, 2011.
- [2] D. H. Titterton and J. L. Weston, *Strapdown Inertial Navigation Technology*, 2nd ed., IET, 2004.
- [3] G. J. Bierman, *Factorization Methods for Discrete Sequential Estimation*, Academic Press, 1977.
- [4] M. S. Grewal and A. P. Andrews, *Kalman Filtering: Theory and Practice Using MATLAB*, 4th ed., Wiley, 2015.
- [5] G. Chang, “Robust Kalman filtering based on Mahalanobis distance as outlier judging criterion,” *Journal of Geodesy*, vol. 88, no. 4, pp. 391–401, 2014.
- [6] P. D. Groves, *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*, 2nd ed., Artech House, 2013.
- [7] J. R. Carpenter and C. N. D’Souza, *Navigation Filter Best Practices*, NASA/TP-2018-219822, NF1676L-29886, 2018. <https://ntrs.nasa.gov/api/citations/20180003657/downloads/20180003657.pdf>